tabela livros

id	titulo	autor
1	Dom Casmurro	Machado de Assis
2	A Hora da Estrela	Clarice Lispector
3	Vidas Secas	Graciliano Ramos

APOSTILA PASSO A PASSO COM TELAS REAIS

Meu Primeiro Banco de Dados

com SQL na prática

Crie bancos e tabelas, insira dados e consulte cada valor, com setas mostrando onde clicar no SQL Lab Workbench

SQL · MySQL · Tabelas · SELECT · JOIN

```
SELECT titulo, preco FROM livros WHERE genero = 'Infantil';
```

Israel Pinheiro Lemos

Professor · Arquiteto de Software Full Stack

Sumário

Os capítulos seguem a ordem em que o banco é construído.

1. O que é um banco de dados? 5

2. Conhecendo a tela do laboratório 8

3. Criando o seu primeiro banco de dados 11

4. Criando as tabelas 15

5. Inserindo dados nas tabelas 22

6. Consultando os dados 26

7. Alterando e apagando dados 37

8. Juntando tabelas com JOIN 43

9. Contando, somando e agrupando 46

10. Histórico e desafios 50

11. Projeto final: o banco da escola 52

Gabarito do projeto final 54

Refazer o banco biblioteca 56

Cola de comandos 58

Glossário 60

Como usar esta apostila

Esta apostila foi feita para quem **nunca construiu um banco de dados**. Você não precisa instalar nada: tudo acontece no **SQL Lab Workbench**, um simulador que funciona direto no navegador e se parece com o MySQL Workbench. Ao final, você terá criado um banco de dados completo, com tabelas ligadas entre si, e saberá consultar cada valor guardado nele.

Abra o laboratório agora, em outra aba do navegador, e deixe-o ao lado deste material:

https://theisraeldev.github.io/sql-lab/

O que cada elemento visual significa



1. Este selo marca o **editor**, onde você digita o comando.
2. Este marca a coluna **titulo** do resultado.
3. Cada **selo numerado** tem uma **seta** que aponta o lugar exato; a **moldura laranja** destaca a área.

Faça junto. Sempre que aparecer esta caixa, é a sua vez: digite o comando no laboratório, exatamente como está escrito, e confira se o seu resultado ficou igual ao da apostila.

Dica. Dicas de quem já passou por isso e deixa o caminho mais fácil.

Cuidado. Pontos em que é fácil errar. Leia antes de apertar o botão.

Pare e pense. Perguntas para você refletir e fixar o que aprendeu.

Erro comum. Erros que todo iniciante comete, com a explicação do que o banco está dizendo.

Regras de ouro para acompanhar

1. **Siga a ordem.** Cada capítulo usa o que foi criado no anterior. Pular etapas gera erros como “a tabela não existe”.
2. **Digite, não copie.** Digitando você memoriza a sintaxe, e o autocompletar do laboratório ensina os nomes.
3. **Não recarregue a página** durante o curso: o laboratório guarda o banco apenas na memória do navegador. Se isso acontecer, basta repetir os comandos dos capítulos 3 a 5 (estão reunidos no apêndice).
4. **Não use o botão “Restaurar”** enquanto estiver construindo o seu banco: ele devolve o laboratório ao estado inicial e apaga o que você criou.
5. **Leia as mensagens do banco.** Quando algo dá errado, a mensagem em vermelho diz o que houve e o que tentar.

Os dados do nosso exemplo (livros e autores) são fictícios e servem apenas para o estudo. Os preços são inventados para facilitar as contas.

CAPÍTULO 1

O que é um banco de dados?

Entenda, sem pressa e sem termos difíceis, o que vamos construir e por que tudo começa pelas tabelas.

1

VOCÊ VAI APRENDER

- ✓ O que é um banco de dados, uma tabela, uma linha e uma coluna
- ✓ Para que serve a chave primária e a chave estrangeira
- ✓ Por que organizamos os dados em tabelas separadas
- ✓ O que é SQL e quais são as suas famílias de comandos

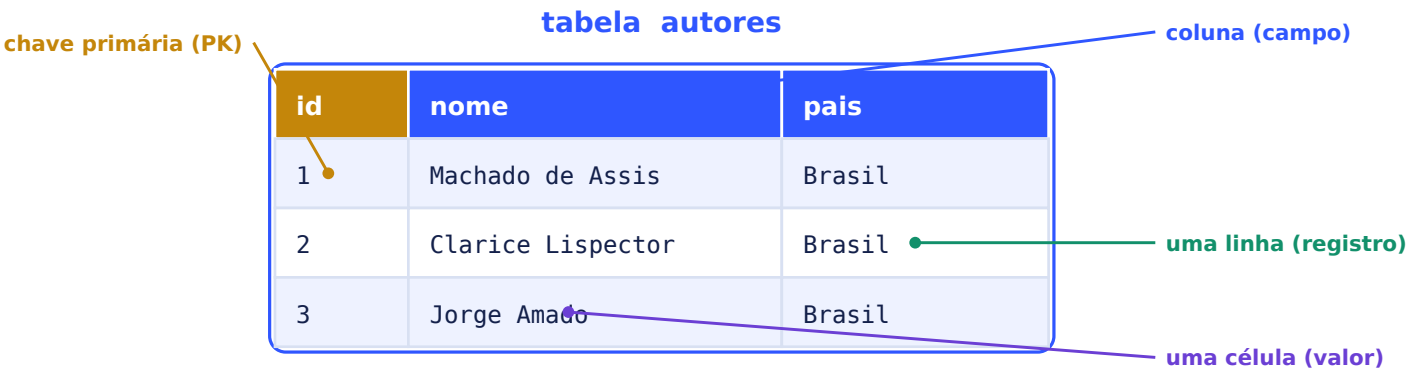
Pense numa biblioteca

Imagine que você cuida de uma biblioteca. Hoje, as informações dos livros estão anotadas em cadernos soltos: um com os títulos, outro com os autores, outro com os preços. Quando alguém pergunta “quais livros do Machado de Assis custam menos de 50 reais?”, você precisa folhear tudo, e ainda corre o risco de escrever o nome do mesmo autor de três maneiras diferentes.

Um **banco de dados** resolve exatamente isso: é um lugar organizado onde guardamos informações de forma estruturada, para que seja rápido, seguro e confiável **guardar, encontrar, corrigir e apagar** qualquer dado. O programa que cuida do banco (o MySQL, por exemplo) se chama **SGBD**: Sistema Gerenciador de Banco de Dados.

Tabela, linha, coluna

Dentro do banco, os dados ficam em **tabelas**. Se você já usou uma planilha, já conhece a ideia: uma tabela tem colunas com nome fixo e cada linha representa uma coisa do mundo real.



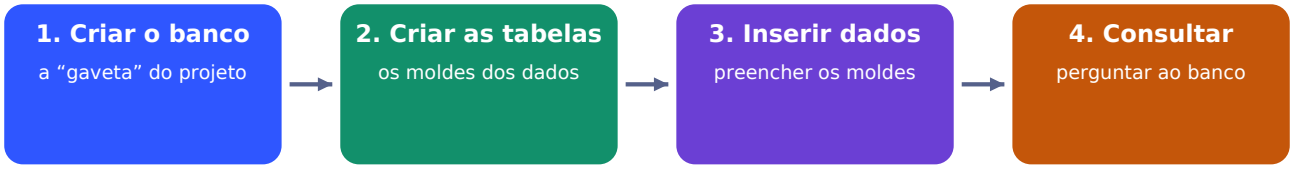
O nome e a ordem das colunas são definidos UMA vez, quando a tabela é criada.

Termo	Significado	No nosso exemplo
Tabela	Um conjunto de dados sobre **um único assunto**.	autores
Coluna (campo)	Uma característica que toda linha possui. Tem nome e tipo.	nome, pais
Linha (registro)	Uma ocorrência completa: um autor, um livro.	2 | Clarice Lispector | Brasil
Célula (valor)	O dado que fica no cruzamento de uma linha com uma coluna.	'Jorge Amado'
Chave primária (PK)	Coluna que identifica cada linha de forma **única**; nunca se repete.	id
Chave estrangeira (FK)	Coluna que guarda o id de uma linha de **outra** tabela, criando o vínculo.	livros.autor_id

Por que criamos tabelas?

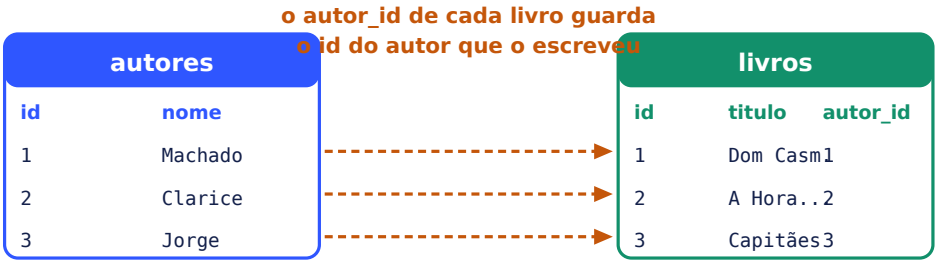
Criar uma tabela é **desenhar o molde** onde os dados vão morar. Antes de guardar qualquer livro, você decide: quais informações importam (título, preço, ano...), de que **tipo** é cada uma (texto, número, valor com centavos) e quais regras valem (o título é obrigatório; o id nunca se repete). Depois disso, o próprio banco passa a **vigiar** essas regras e recusa dados errados.

A tabela é o contrato entre você e o banco. Por isso a ordem do trabalho é sempre a mesma:



Por que não colocar tudo numa tabela só?

Se cada livro guardasse o nome do autor, “Machado de Assis” apareceria em vários livros. Um erro de digitação criaria um “autor” diferente, e corrigir um nome exigiria mexer em várias linhas. A solução é **separar assuntos**: uma tabela para autores, outra para livros, ligadas por um número.



Assim, cada autor é cadastrado **uma vez só**, e os livros apenas apontam para ele. É essa ligação que o comando **JOIN**, no capítulo 8, usa para juntar as informações na hora da consulta.

E o SQL?

SQL (Structured Query Language, “linguagem de consulta estruturada”) é a língua que usamos para conversar com o banco. Seus comandos se agrupam em famílias:

Família	Para que serve	Comandos que veremos
DDL — definição	Criar e alterar a **estrutura** (bancos e tabelas).	CREATE, ALTER, DROP
DML — manipulação	Mexer nos **dados** dentro das tabelas.	INSERT, UPDATE, DELETE
DQL — consulta	**Perguntar** ao banco, sem alterar nada.	SELECT
Comandos de apoio	Ver o que existe e escolher onde trabalhar.	SHOW, DESCRIBE, USE

> **Pare e pense.** Se você fosse montar a tabela **alunos** de uma escola, quais colunas ela teria? E qual delas seria a chave primária? Anote: você vai usar essa ideia no projeto final.

CAPÍTULO 2

Conhecendo a tela do laboratório

Um passeio pelos oito pontos da tela que você vai usar o tempo todo.

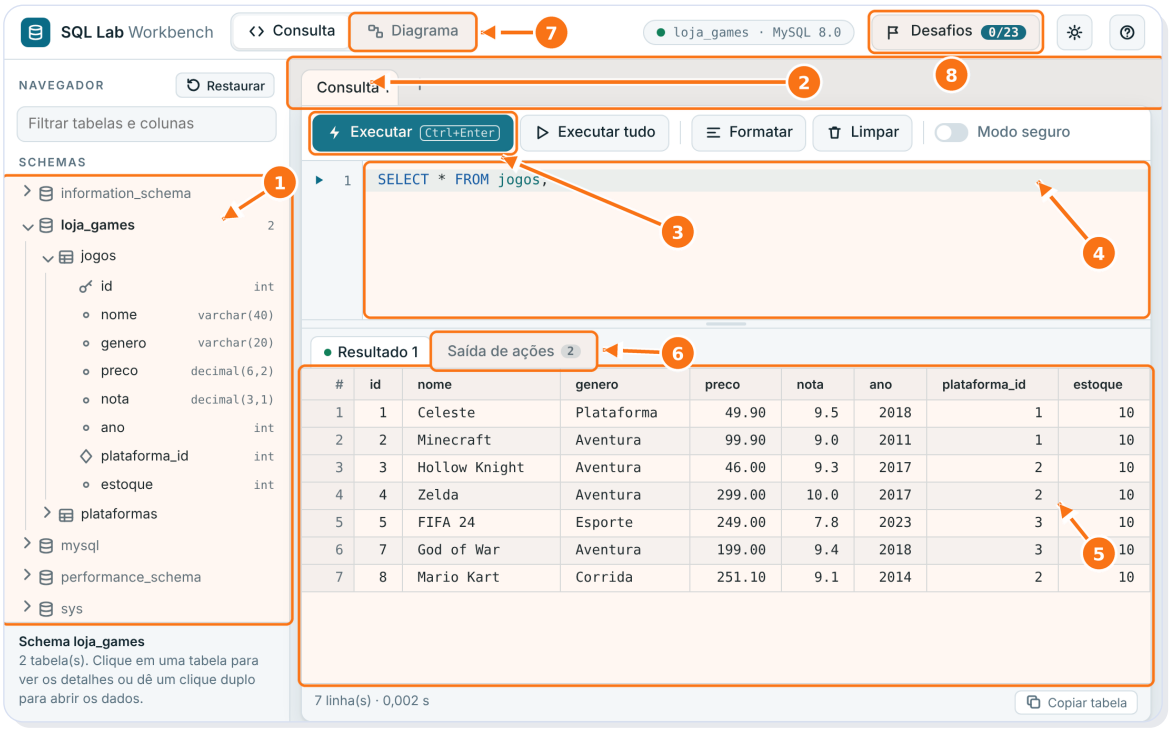
VOCÊ VAI APRENDER

- ✓ Onde digitar e onde aparece o resultado
- ✓ Para que serve cada botão da barra de ferramentas
- ✓ Como o Navegador mostra bancos, tabelas e colunas
- ✓ Os atalhos de teclado mais importantes

Ao abrir o laboratório pela primeira vez, você vê um banco de exemplo chamado **loja_games**. Ele serve só para você se acostumar: o **nosso** banco será criado do zero a partir do próximo capítulo. Vamos conhecer a tela:

Faça junto. Abra o laboratório e localize, na sua tela, cada um dos oito itens abaixo.

VISÃO GERAL



1. **Navegador** (lado esquerdo): lista os bancos (schemas). Clique na setinha de um banco para abrir as tabelas e, depois, as colunas.
2. **Abas de consulta**: cada aba é uma folha de rascunho. O “+” abre outra aba.
3. **Executar**: roda o comando em que o cursor está. Atalho: **Ctrl+Enter**.
4. **Editor**: é aqui que você digita o SQL. As palavras ganham cor sozinhas.
5. **Área de resultado**: mostra a tabela devolvida pela consulta ou a mensagem de sucesso ou erro.
6. **Saída de ações**: o histórico de tudo o que você executou, com hora e duração.
7. **Diagrama**: desenha as tabelas e as ligações entre elas.
8. **Desafios**: perguntas prontas, com correção automática.

A barra de ferramentas

Botão	O que faz
Executar (Ctrl+Enter)	Executa **apenas um comando**: o que está sob o cursor (ou o trecho selecionado).
Executar tudo (Ctrl+Shift+Enter)	Executa **todos** os comandos do editor, de cima para baixo.
Formatar	Arruma o texto e põe as palavras do SQL em maiúsculas.
Limpar	Apaga o texto da aba atual.
Modo seguro	Impede UPDATE e DELETE sem WHERE em uma coluna-chave. Veremos no capítulo 7.
Restaurar (no Navegador)	Devolve o laboratório ao estado inicial. **Apaga o que você criou.**

Importante. Existe uma diferença importante entre **Executar** e **Executar tudo**. Se o editor tiver três comandos e você clicar em Executar, **só um** deles roda: aquele em que o cursor está. Para rodar vários de uma vez, use Executar tudo.

Dica. Ao lado do número de cada linha aparece um triângulo ▶. Clicar nele executa aquele comando específico. É útil quando você tem vários comandos na mesma aba.

Regras de escrita do SQL

- Todo comando termina com **ponto e vírgula (;)**.
- Os comandos podem ocupar várias linhas: o **;** é que marca o fim.
- Palavras do SQL (SELECT, FROM...) podem ser maiúsculas ou minúsculas. Usamos maiúsculas por tradição, para destacar.
- Textos vão entre **aspas simples**: 'Romance'. Números não levam aspas: 1899.
- Números decimais usam **ponto**, não vírgula: 39.90.
- Comentários começam com dois hífens e um espaço (`-- isto é um comentário`) e são ignorados.

CAPÍTULO 3

Criando o seu primeiro banco de dados

Três comandos: ver o que existe, criar o banco da biblioteca e entrar nele.

VOCÊ VAI APRENDER

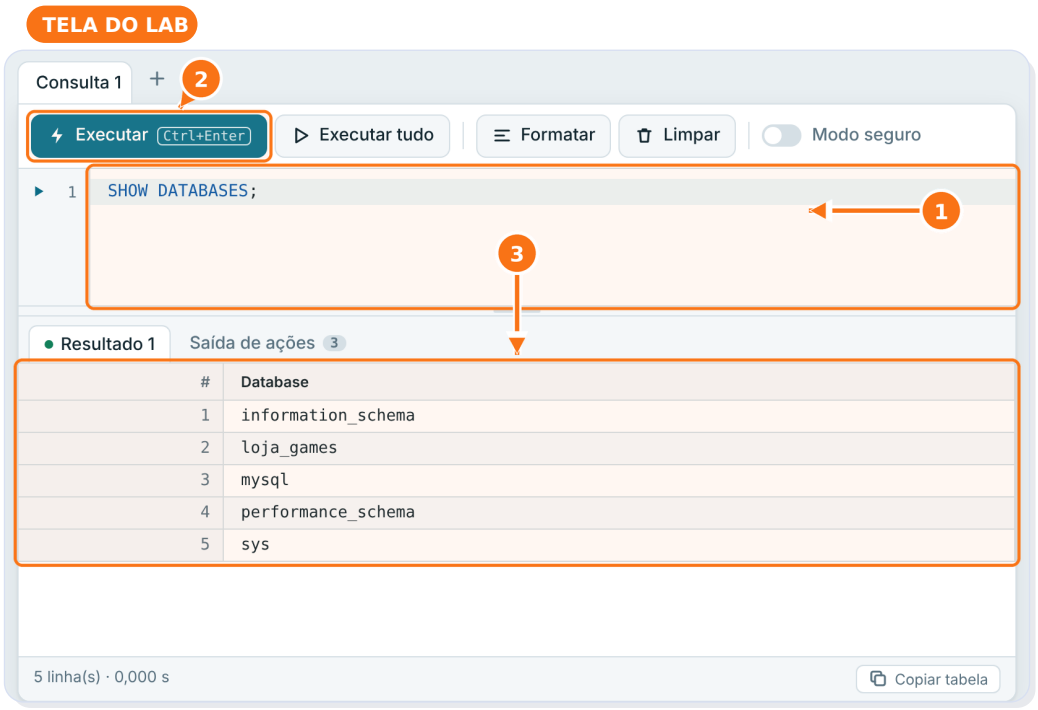
- ✓ Listar os bancos com SHOW DATABASES
- ✓ Criar um banco com CREATE DATABASE
- ✓ Escolher onde trabalhar com USE
- ✓ Como o Navegador mostra o banco novo

1 Ver o que já existe: SHOW DATABASES

O que faz: mostra a lista de bancos de dados disponíveis no servidor. **Para que serve:** antes de criar algo, é bom saber o que já existe, para não repetir nomes.

```sql
SHOW DATABASES;
```

Faça junto. Clique no editor, digite o comando acima e aperte **Ctrl+Enter** (ou clique em Executar).



1. Digite o comando no **editor**.
2. Clique em **Executar** (ou use Ctrl+Enter).
3. A **lista de bancos** aparece na área de resultado. Ainda não existe nenhum “biblioteca”.

RESULTADO ESPERADO

Database
information_schema
loja_games
mysql
performance_schema
sys

> **Importante.** **information_schema**, **mysql**, **performance_schema** e **sys** são bancos do próprio sistema. Eles guardam informações internas. Nunca apague nem altere esses bancos.

2 Criar o banco: CREATE DATABASE

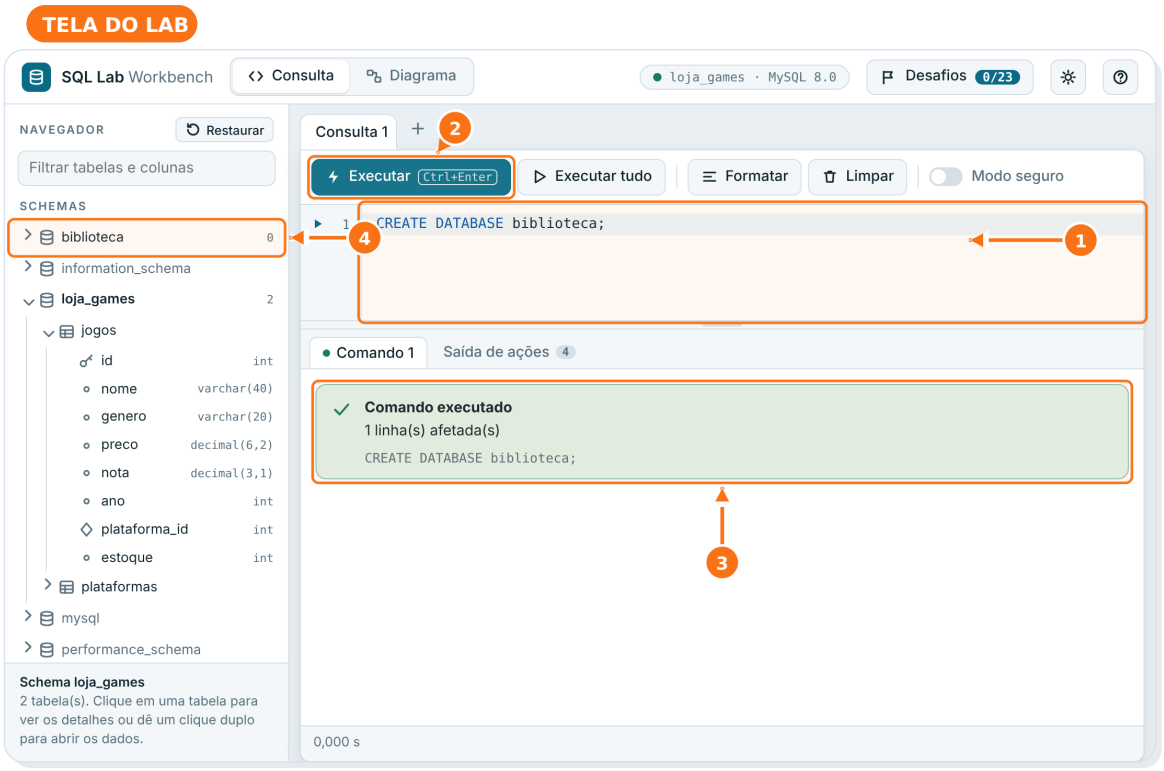
O que faz: cria um banco novo e vazio. **Para que serve:** cada projeto ganha o seu banco, uma espécie de gaveta onde ficarão as tabelas daquele sistema. Os livros da biblioteca não se misturam com a loja de games.

```sql
CREATE DATABASE biblioteca;
```

CADA PEDAÇO DO COMANDO

Pedaço	O que significa
`CREATE DATABASE`	“criar banco de dados”: o tipo de coisa que será criada.
`biblioteca`	o **nome** que você escolheu. Use letras minúsculas, sem espaços e sem acentos.
`;`	fecha o comando.

Faça junto. Limpe o editor (botão **Limpar**), digite o comando e execute.



1. Digite o comando.
2. Execute.
3. A faixa verde confirma: **Comando executado**, 1 linha afetada.
4. O banco **biblioteca** aparece no Navegador, com 0 tabelas.

Erro comum. Se aparecer **Erro 1007** (“ele já existe”), o banco já foi criado — provavelmente você executou duas vezes. Não precisa criar de novo: siga para o passo seguinte. (Uma forma segura de evitar isso é `CREATE DATABASE IF NOT EXISTS biblioteca;`.)

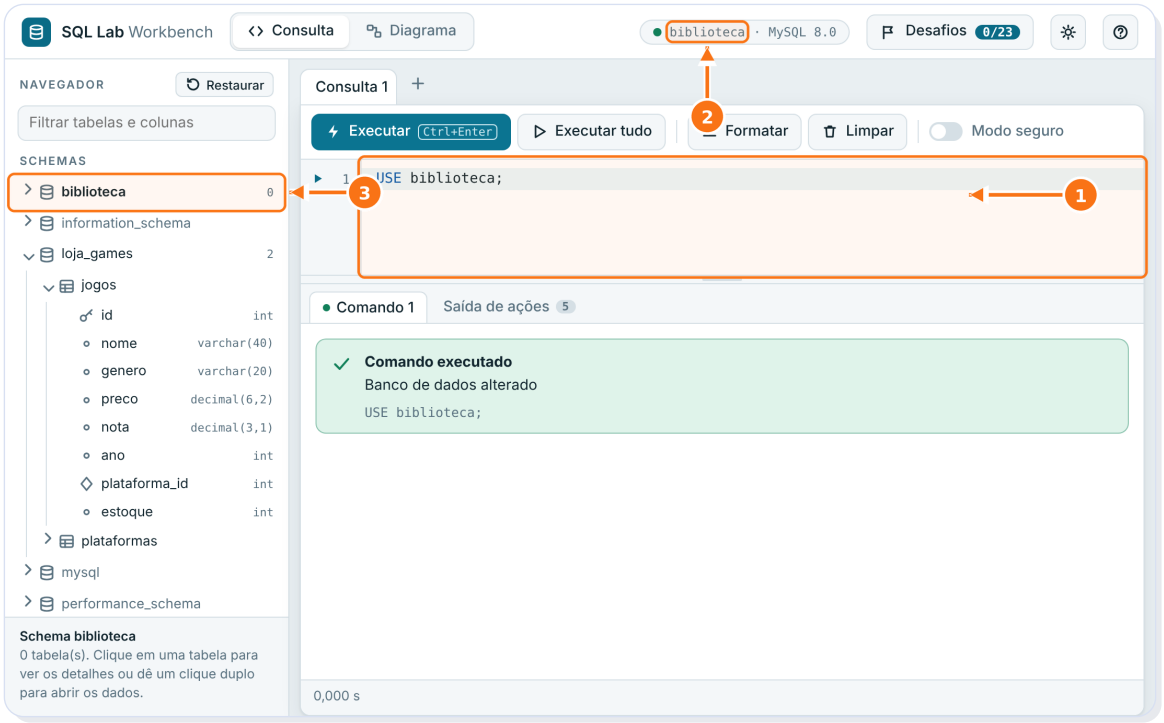
3 Entrar no banco: USE

O que faz: escolhe o banco com o qual você vai trabalhar. **Para que serve:** o servidor guarda vários bancos; sem o USE, ele não sabe em qual deles criar a tabela. É como abrir a gaveta certa antes de guardar um papel.

```sql
USE biblioteca;
```

Faça junto. Digite e execute. Depois, olhe no topo da tela: o nome do banco ativo muda.

TELA DO LAB



1. Digite **USE biblioteca;**.
2. O topo da tela passa a mostrar **biblioteca** como banco ativo.
3. No Navegador, o banco em uso fica em **negrito**.

Cuidado. Toda vez que você abrir o laboratório de novo, lembre-se do USE. Se esquecer, aparece o **Erro 1046** (“Nenhum banco selecionado”) ou o comando atua no banco errado.

Pare e pense. Qual é a diferença entre **CREATE DATABASE** e **USE**? Um constrói a gaveta; o outro abre a gaveta.

CAPÍTULO 4

Criando as tabelas

Agora o coração do projeto: desenhar os moldes que vão guardar os autores e os livros.

4

VOCÊ VAI APRENDER

- ✓ Como o CREATE TABLE gera uma tabela, passo a passo
- ✓ Os tipos de dados: INT, VARCHAR, DECIMAL, DATE
- ✓ NOT NULL, AUTO_INCREMENT, PRIMARY KEY e FOREIGN KEY
- ✓ Conferir o resultado com SHOW TABLES, DESCRIBE e o Diagrama

O que acontece quando você cria uma tabela?

Ao executar um CREATE TABLE, o banco faz cinco coisas, nesta ordem:

1. **Lê** o comando e confere se a sintaxe está correta.
2. **Confere** se já não existe uma tabela com esse nome no banco atual.
3. **Registra a estrutura** (nome, colunas, tipos e regras) no “catálogo” do banco: a ficha técnica da tabela.
4. **Reserva o espaço** para as linhas futuras.
5. **Devolve** a mensagem de sucesso. A tabela nasce **vazia**: tem forma, mas nenhum dado.

Tipos de dados: o que cabe em cada coluna

Cada coluna precisa de um **tipo**. Ele define o que pode ser guardado e quanto espaço ocupa. O banco recusa qualquer valor que não combine com o tipo.

Tipo	Guarda	Exemplo	Quando usar
`INT`	números inteiros	`1899, 256`	ids, anos, quantidades
`VARCHAR(n)`	texto de até n letras	`'Dom Casmurro'`	nomes, títulos, e-mails
`DECIMAL(p,s)`	números com casas fixas: p dígitos no total, s depois da vírgula	`39.90`	dinheiro, notas
`DATE`	uma data (ano-mês-dia)	`'2026-03-15'`	nascimento, empréstimo

Dica. **DECIMAL(6,2)** significa: até 6 dígitos no total, 2 deles depois do ponto. Cabe de 0.00 até 9999.99. Para dinheiro, prefira DECIMAL: ele não perde centavos nas contas.

4 Criar a tabela autores

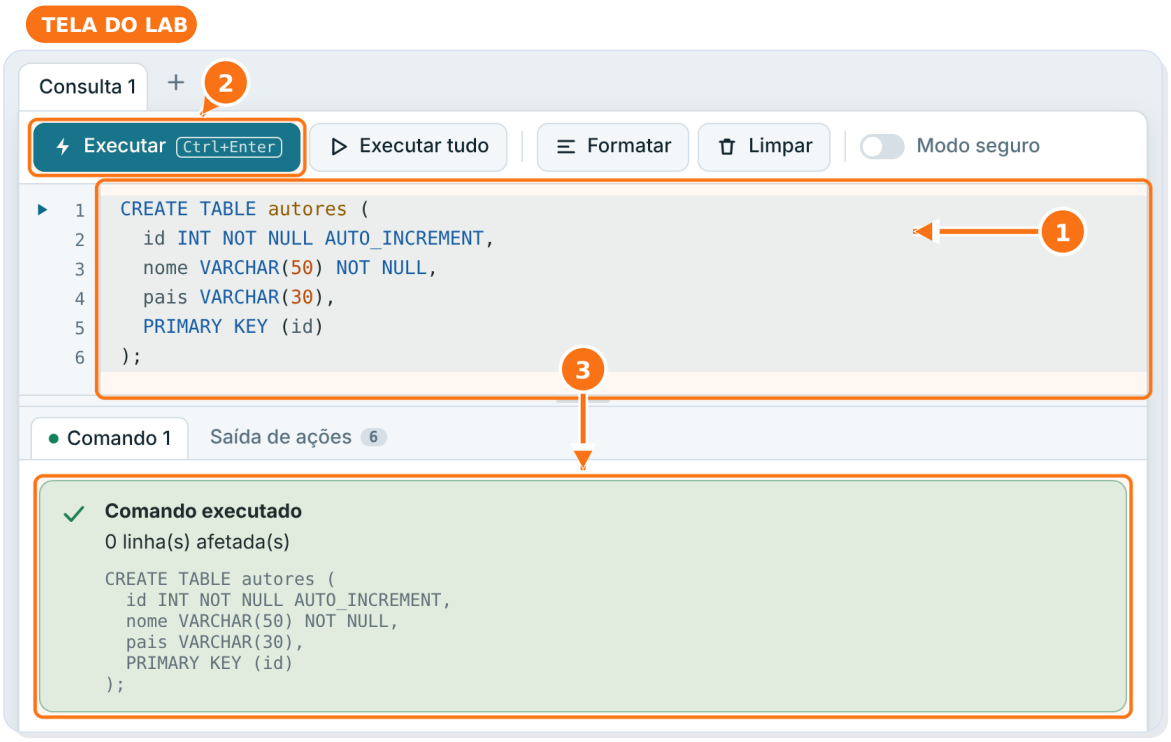
Começamos pelos **autores** porque a tabela de livros vai depender dela. **Regra:** uma tabela que é apontada por outra precisa existir primeiro.

```SQL
CREATE TABLE autores (
  id INT NOT NULL AUTO_INCREMENT,
  nome VARCHAR(50) NOT NULL,
  pais VARCHAR(30),
  PRIMARY KEY (id)
);
```

CADA PEDAÇO DO COMANDO

Pedaço	O que significa
`CREATE TABLE autores`	criar uma tabela chamada **autores**. Os parênteses guardam a lista de colunas.
`id INT`	coluna **id** do tipo número inteiro.
`NOT NULL`	a coluna é **obrigatória**: não aceita valor vazio.
`AUTO_INCREMENT`	o banco numera sozinho: 1, 2, 3... Você nunca precisa digitar o id.
`nome VARCHAR(50)`	coluna de texto com até 50 caracteres; obrigatória.
`pais VARCHAR(30)`	texto de até 30 caracteres; sem NOT NULL, então pode ficar vazio (NULL).
`PRIMARY KEY (id)`	declara **id** como chave primária: única e nunca vazia.
`, (vírgula)`	separa uma coluna da outra. **A última não leva vírgula.**

Faça junto. Digite o comando inteiro, em várias linhas, como está acima. Para quebrar a linha, aperte Enter. O cursor precisa estar dentro do comando quando você executar.



1. Digite o comando dentro do **editor**.
2. Clique em **Executar**.
3. **Comando executado**: a tabela foi criada. “0 linhas afetadas” é normal: nenhum dado foi mexido.

Erro comum. **Erro 1064 (sintaxe)** logo após criar? Procure uma vírgula sobrando antes do parêntese final, um parêntese esquecido ou o ponto e vírgula faltando.

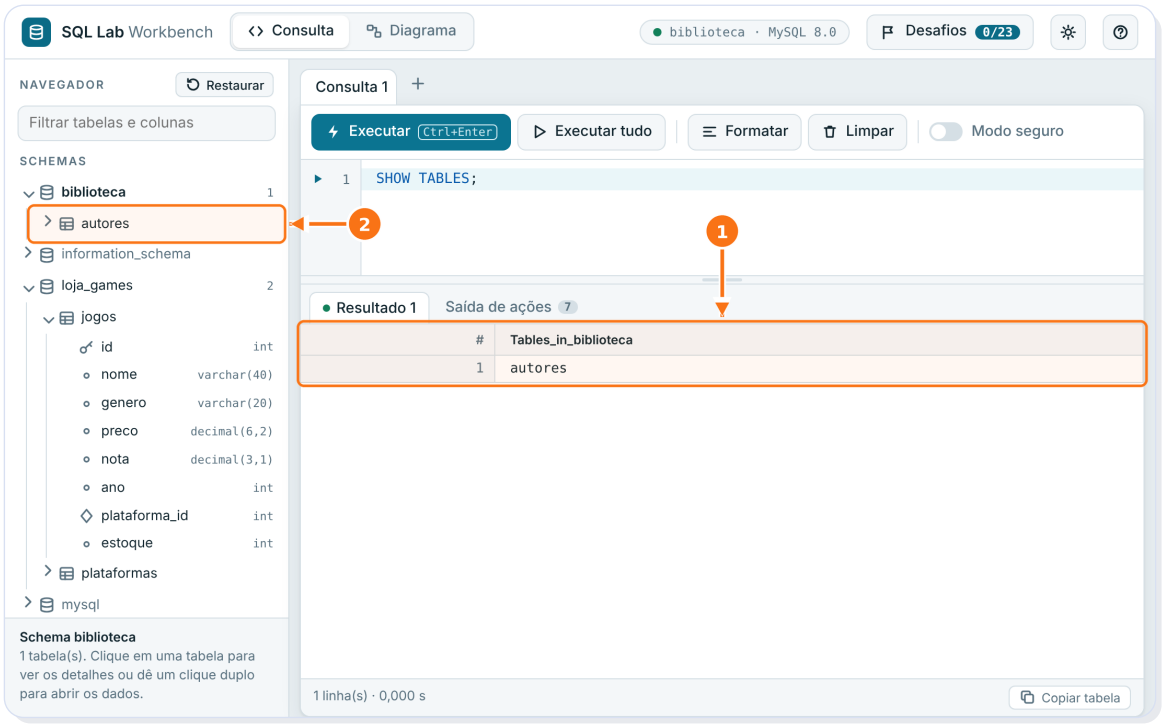
5 Conferir: SHOW TABLES

O que faz: lista as tabelas do banco em uso. **Para que serve:** confirmar que a criação funcionou.

```sql
SHOW TABLES;
```

Faça junto. Execute. Depois, no Navegador, clique na setinha ao lado de **biblioteca** para abrir o banco.

TELA DO LAB



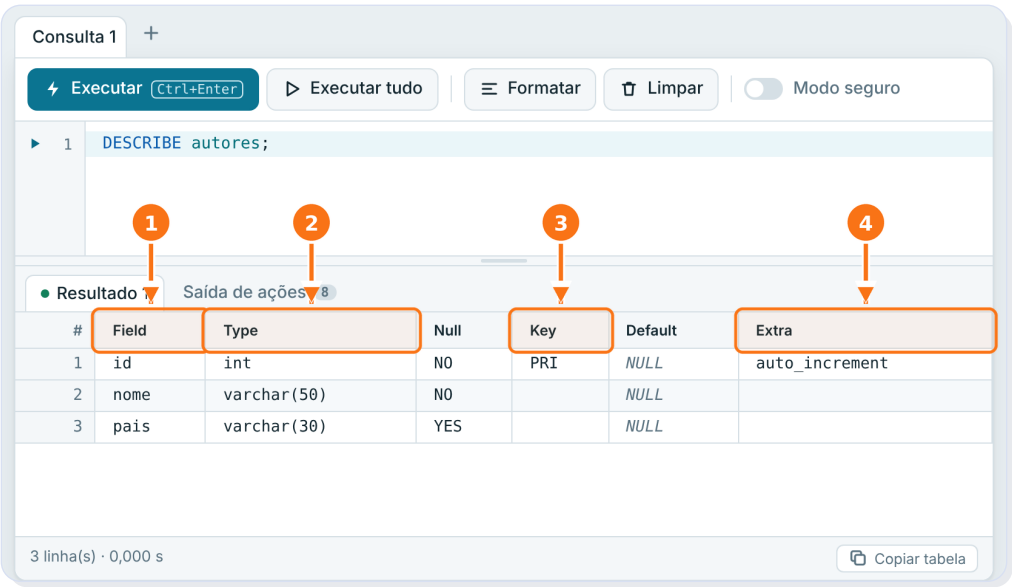
1. O resultado lista **autores**, a única tabela criada até agora.
2. No Navegador, a tabela aparece dentro do banco **biblioteca**.

6 Ver a estrutura: DESCRIBE

O que faz: mostra as colunas da tabela, com tipo e regras. **Para que serve:** relembrar como a tabela foi montada, sem precisar reler o CREATE TABLE.

```sql
DESCRIBE autores;
```

TELA DO LAB



1. **Field**: o nome de cada coluna.
2. **Type**: o tipo de dado (int, varchar(50)...).
3. **Key**: PRI marca a chave primária.
4. **Extra**: auto_increment, a numeração automática.

RESULTADO ESPERADO

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
nome	varchar(50)	NO		NULL	
pais	varchar(30)	YES		NULL	

A coluna **Null** diz se a coluna aceita valor vazio: NO é obrigatória, YES é opcional.

7 Criar a tabela livros (com chave estrangeira)

Agora a tabela principal. Ela tem uma coluna nova, **autor_id**, que será a ligação com a tabela autores:

SQL

```sql
CREATE TABLE livros (
  id INT NOT NULL AUTO_INCREMENT,
  titulo VARCHAR(80) NOT NULL,
  genero VARCHAR(20),
  preco DECIMAL(6,2),
  ano INT,
  paginas INT,
  autor_id INT,
  PRIMARY KEY (id),
  FOREIGN KEY (autor_id) REFERENCES autores(id)
);
```

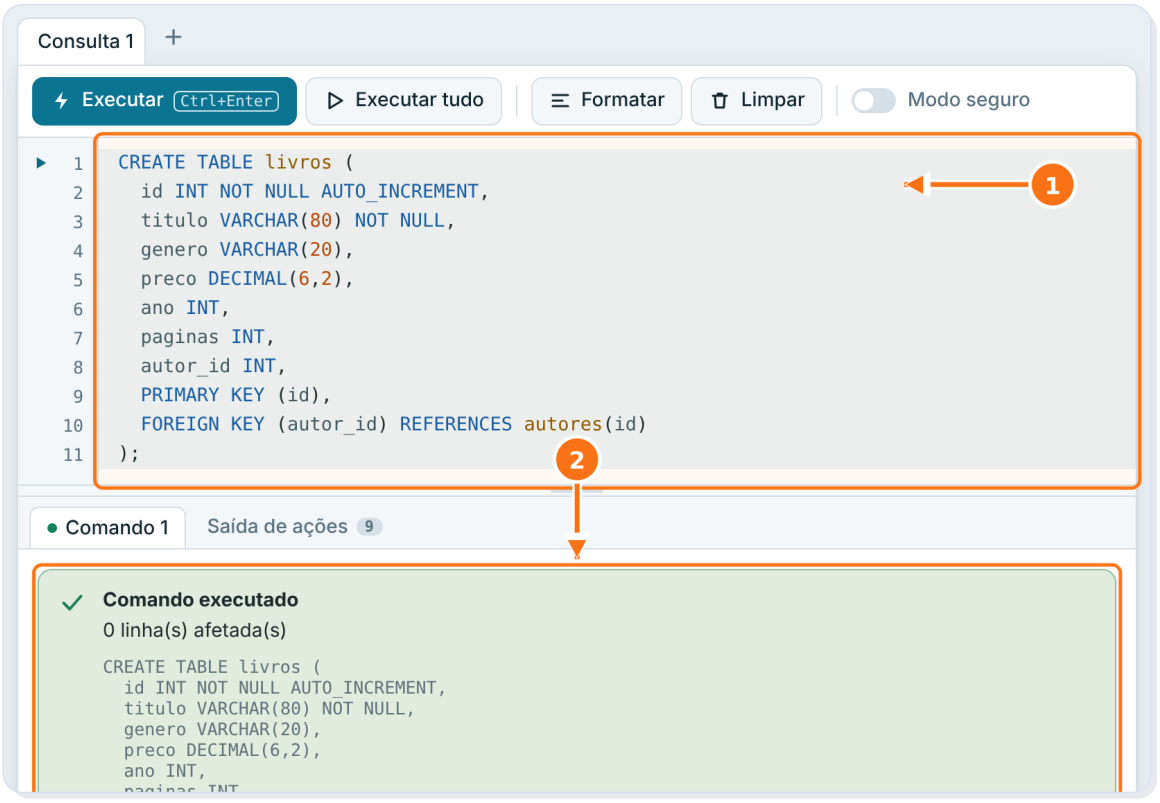
CADA PEDAÇO DO COMANDO

Pedaço	O que significa
`titulo VARCHAR(80) NOT NULL`	todo livro precisa de um título.
`genero VARCHAR(20)`	gênero opcional (Romance, Infantil...).
`preco DECIMAL(6,2)`	preço com 2 casas decimais.
`ano INT, paginas INT`	números inteiros: ano de publicação e número de páginas.
`autor_id INT`	guardará o **id** do autor. É o "fio" que liga o livro ao autor.
`FOREIGN KEY (autor_id)`	declara que **autor_id** é uma chave estrangeira...
`REFERENCES autores(id)`	...que aponta para a coluna **id** da tabela **autores**.

Importante. A FOREIGN KEY é uma **proteção**. Com ela, o banco impede que você cadastre um livro de um autor que não existe e impede apagar um autor que ainda tem livros. Veremos os dois casos nos capítulos 5 e 7.

Faça junto. Digite o comando CREATE TABLE livros completo e execute. Depois rode **SHOW TABLES;** e confira que agora há duas tabelas.

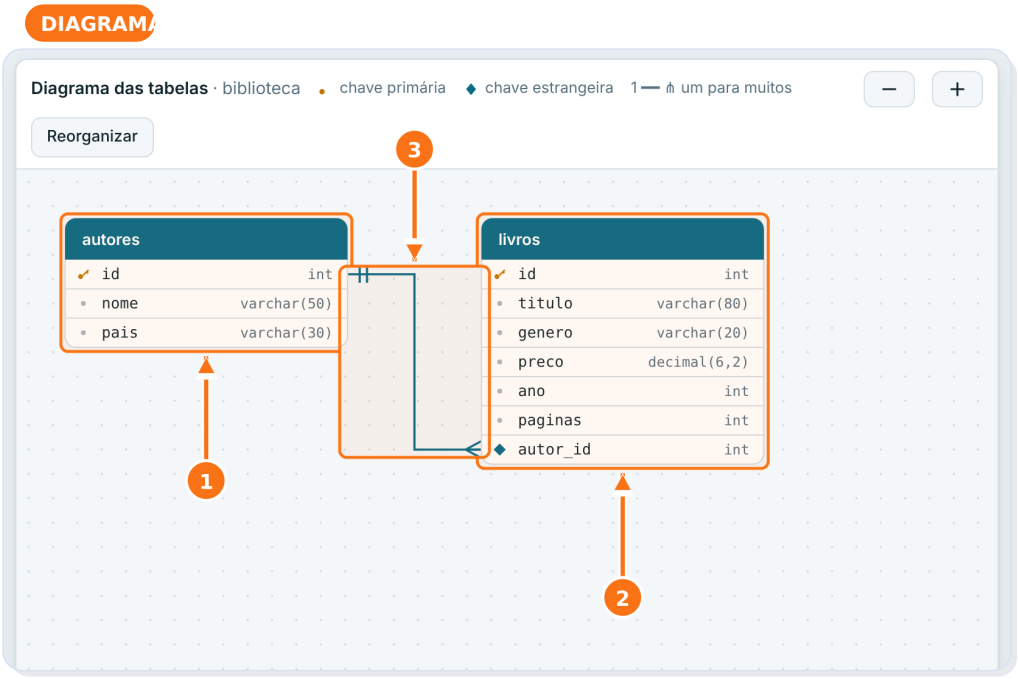
TELA DO LAB



1. Todo o comando fica no editor, em várias linhas.
2. A mensagem verde confirma a criação da tabela.

8 Ver o desenho: o Diagrama

Clique na aba **Diagrama**, no topo da tela. O laboratório desenha as tabelas e a ligação entre elas, no estilo usado por profissionais (diagrama entidade-relacionamento).



1. Tabela **autores**: a chave dourada marca a chave primária.
2. Tabela **livros**: o losango azul marca **autor_id**, a chave estrangeira.
3. A **linha de ligação**: um autor (||) para muitos livros (pé-de-galinha).

Pare e pense. Leia a ligação em voz alta: *um* autor pode ter *muitos* livros, mas cada livro tem *um* só autor. Esse tipo de relação se chama **um-para-muitos** e é a mais comum nos bancos de dados.

Dica. Volte para a aba **Consulta** para continuar. Dá para arrastar as tabelas do diagrama e usar **Reorganizar** para arrumá-las.

CAPÍTULO 5

Inserindo dados nas tabelas

As tabelas existem, mas estão vazias. Hora de cadastrar autores e livros.

5

VOCÊ VAI APRENDER

- ✓ Usar INSERT INTO para guardar linhas
- ✓ Por que listar as colunas e o que é NULL
- ✓ Inserir várias linhas de uma vez
- ✓ Respeitar a ordem imposta pela chave estrangeira

9 Cadastrar os autores

O que faz: o INSERT adiciona linhas novas a uma tabela. **Para que serve:** é assim que os dados entram no banco. Cada linha inserida passa a ser um registro.

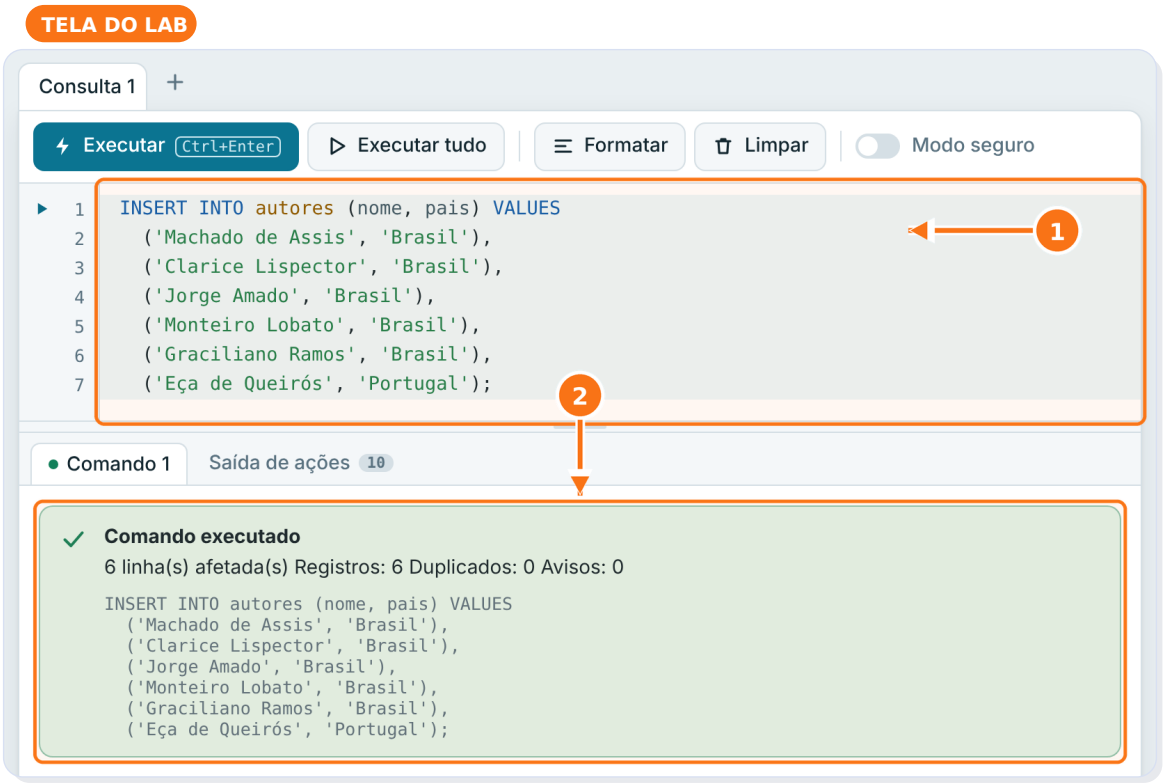
```sql
INSERT INTO autores (nome, pais) VALUES
  ('Machado de Assis', 'Brasil'),
  ('Clarice Lispector', 'Brasil'),
  ('Jorge Amado', 'Brasil'),
  ('Monteiro Lobato', 'Brasil'),
  ('Graciliano Ramos', 'Brasil'),
  ('Eça de Queirós', 'Portugal');
```

CADA PEDAÇO DO COMANDO

Pedaço	O que significa
`INSERT INTO autores`	“inserir em autores”: informa a tabela de destino.
`(nome, pais)`	a lista das colunas que serão preenchidas, **na ordem** em que os valores virão.
`VALUES`	“valores”: o que vem a seguir são os dados.
`('Machado de Assis', 'Brasil')`	uma linha. Texto entre aspas simples, na mesma ordem das colunas.

Pedaço	O que significa
`, entre as linhas`	separa as linhas. Assim inserimos **seis autores com um único comando**.
`(id ausente)`	não informamos o **id**: o AUTO_INCREMENT numera de 1 a 6.

Faça junto. Digite o INSERT completo e execute. Confira a mensagem: **6 linhas afetadas**.



1. Os seis autores no editor, uma linha para cada um.
2. **6 linha(s) afetada(s)** e “Registros: 6”: seis linhas entraram.

10 Cadastrar os livros

Os livros usam o **autor_id** para apontar para o autor. O número 1 é Machado de Assis, o 2 é Clarice Lispector, e assim por diante, na ordem em que os autores foram inseridos.

```sql
INSERT INTO livros (titulo, genero, preco, ano, paginas, autor_id) VALUES
  ('Dom Casmurro', 'Romance', 39.90, 1899, 256, 1),
  ('Memórias Póstumas de Brás Cubas', 'Romance', 44.90, 1881, 368, 1),
  ('A Hora da Estrela', 'Romance', 34.50, 1977, 88, 2),
  ('Capitães da Areia', 'Romance', 42.00, 1937, 280, 3),
  ('Gabriela, Cravo e Canela', 'Romance', 59.90, 1958, 424, 3),
  ('Reinações de Narizinho', 'Infantil', 29.90, 1931, 192, 4),
  ('Vidas Secas', 'Romance', 36.00, 1938, NULL, 5),
  ('O Primo Basílio', 'Romance', 41.50, 1878, 400, 6),
  ('Caçadas de Pedrinho', 'Infantil', 27.90, 1933, 120, 4);
```

Faça junto. Digite os nove livros e execute. A mensagem deve ser **9 linhas afetadas**.



1. Nove livros num só comando.
2. **9 linha(s) afetada(s)**: tudo certo.

Pontos para observar

- **Texto** vai entre aspas simples; **números** não (1899, 39.90).
- **NULL** significa “sem valor”. Em *Vidas Secas* deixamos as páginas como NULL porque não sabemos o número. NULL não é zero nem texto vazio: é a ausência de informação.
- Quem tem **NOT NULL** (como o título) **precisa** de valor, senão o banco recusa o comando.
- **Aspas dentro de texto:** se o texto tiver apóstrofo, repita-o: 'D''Água'.

Quando o banco recusa um dado

As regras que você definiu na criação da tabela agora entram em ação. Veja três erros que você pode provocar de propósito, para entender a proteção (não precisa digitar; apenas leia):

Comando	Erro	Por quê
`INSERT INTO livros (titulo, autor_id) VALUES ('Teste', 99);`	1452	O autor 99 não existe: a FOREIGN KEY barra o livro “órfão”.
`INSERT INTO livros (genero) VALUES ('Romance');`	1364	O título é NOT NULL e não recebeu valor.
`INSERT INTO autores (id, nome) VALUES (1, 'Repetido');`	1062	O id 1 já existe: a chave primária não pode repetir.

Importante. Por causa da chave estrangeira, **insira sempre primeiro a tabela “pai”** (autores) e só depois a “filha” (livros). Foi por isso que cadastramos os autores antes dos livros.

Pare e pense. Por que o banco recusa o livro com autor_id = 99? Imagine um livro cujo autor não existe: qualquer relatório com o nome do autor daria resultado vazio ou errado.

CAPÍTULO 6

Consultando os dados

O capítulo mais usado da vida de quem trabalha com bancos: perguntar e receber respostas.

6

VOCÊ VAI APRENDER

- ✓ SELECT para ver tudo ou só algumas colunas
- ✓ Filtrar linhas com WHERE, AND, OR, IN, BETWEEN, LIKE e IS NULL
- ✓ Ordenar com ORDER BY e limitar com LIMIT
- ✓ Usar o autocompletar, o ▶ do editor e ler mensagens de erro

11 Ver tudo: SELECT * FROM

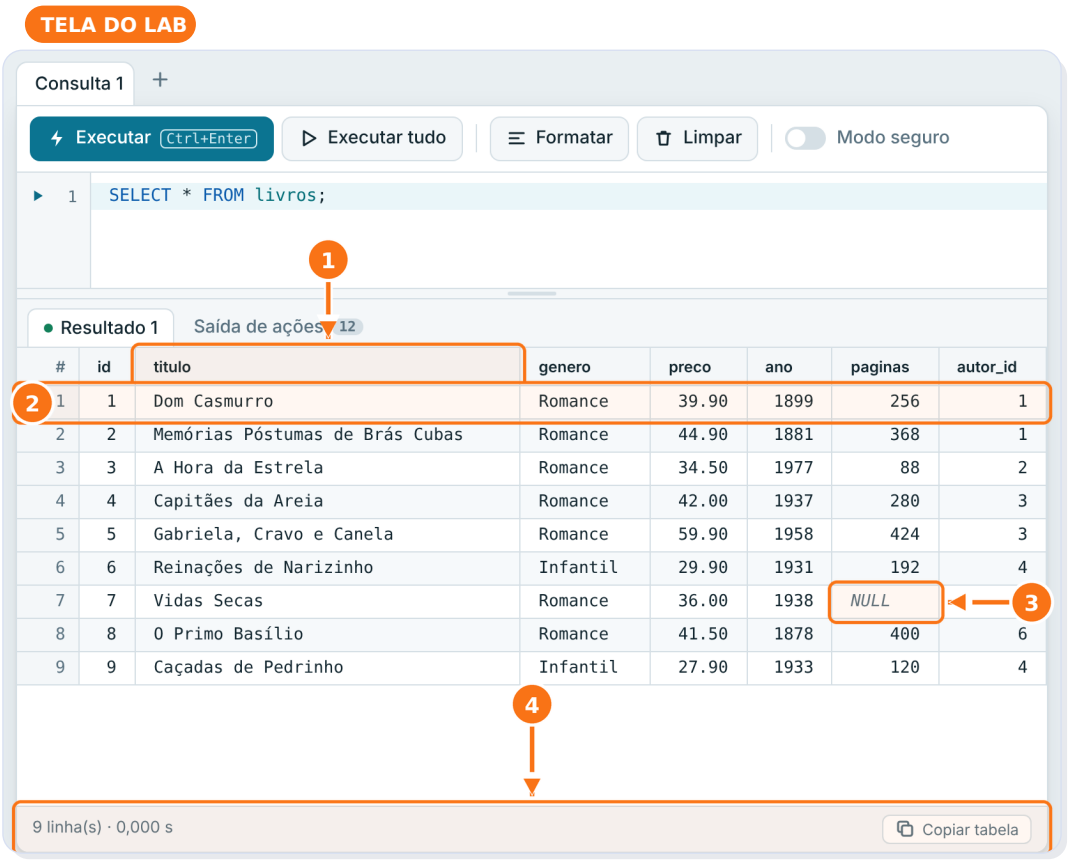
O que faz: o SELECT consulta uma tabela e devolve linhas. **Para que serve:** é a única forma de ler os dados guardados. O asterisco * significa “todas as colunas”.

```sql
SELECT * FROM livros;
```

CADA PEDAÇO DO COMANDO

Pedaço	O que significa
`SELECT`	“selecione”: diz que queremos **ler** dados.
`*`	todas as colunas da tabela.
`FROM livros`	“a partir de livros”: de qual tabela vêm os dados.

Faça junto. Digite e execute. Compare o seu resultado com a tela abaixo.

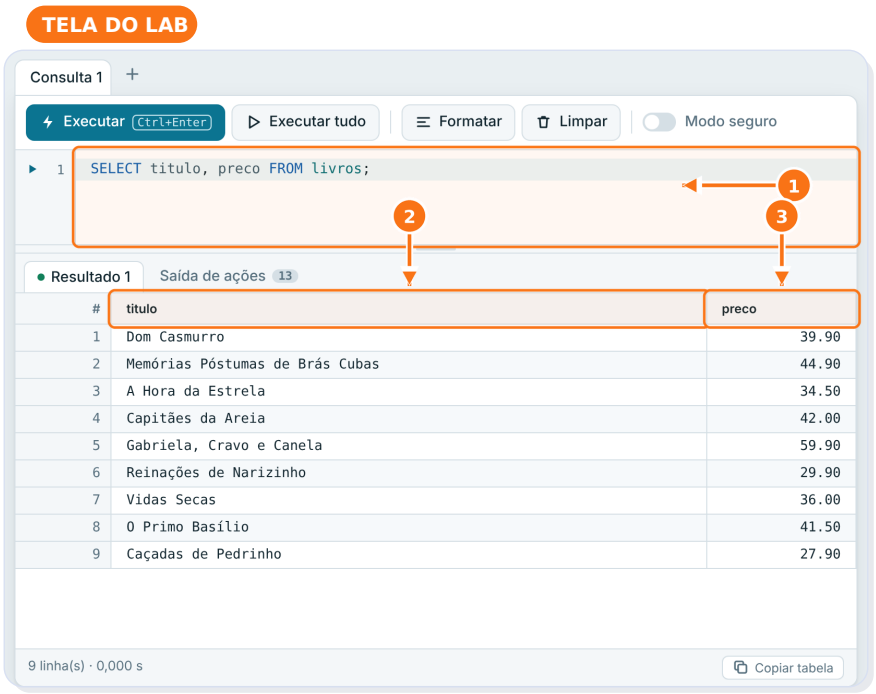


1. Os **nomes das colunas** formam o cabeçalho.
2. Cada **linha** é um livro cadastrado.
3. **NULL** aparece em itálico: Vidas Secas está sem número de páginas.
4. Rodapé: **9 linhas** e o tempo da consulta.

12 Escolher colunas

Quase nunca queremos todas as colunas. Para pedir só algumas, liste os nomes separados por vírgula, na ordem em que deseja ver:

```sql
SELECT titulo, preco FROM livros;
```



1. O editor com as duas colunas escolhidas.
2. Coluna **titulo**.
3. Coluna **preco**.

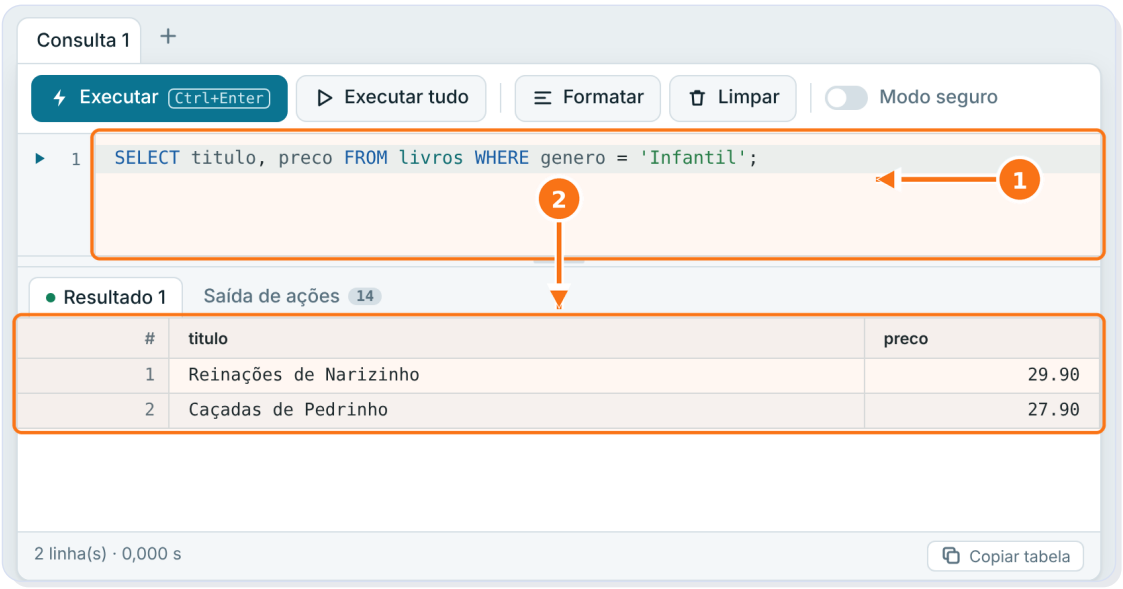
> **Dica.** Pedir só o necessário deixa a consulta mais rápida e o resultado mais fácil de ler. Em bancos grandes, usar **SELECT *** sem necessidade é considerado má prática.

13 Filtrar linhas: WHERE

O que faz: o WHERE mantém apenas as linhas que atendem a uma condição. **Para que serve:** responder perguntas específicas: “quais livros são infantis?”

```sql
SELECT titulo, preco FROM livros WHERE genero = 'Infantil';
```

TELA DO LAB



1. A condição vem depois do **WHERE**.
2. Só os livros do gênero Infantil voltaram: **2 linhas**.

RESULTADO ESPERADO

titulo	preco
Reinações de Narizinho	29.90
Caçadas de Pedrinho	27.90

Operadores de comparação

Operador	Significa	Exemplo
`=`	igual a	`genero = 'Romance'`
`<> ou !=`	diferente de	`pais <> 'Brasil'`
`> e <`	maior que / menor que	`preco > 40`
`>= e <=`	maior ou igual / menor ou igual	`paginas >= 300`
`BETWEEN a AND b`	entre dois valores (inclusive)	`preco BETWEEN 30 AND 40`
`IN (...)`	igual a algum item da lista	`autor_id IN (1, 3)`
`LIKE`	texto parecido com um padrão (% = qualquer coisa)	`titulo LIKE 'C%'`
`IS NULL`	o campo está vazio	`paginas IS NULL`

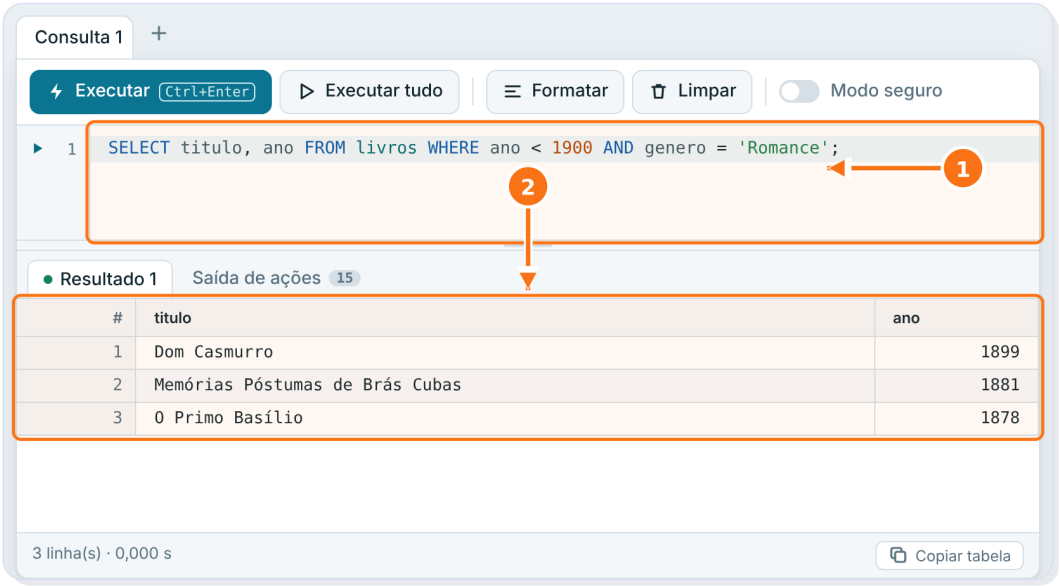
Erro comum. Para testar se um campo é vazio use **IS NULL**, nunca = NULL. NULL não é um valor, então a comparação com = nunca dá verdadeiro.

14 Combinar condições: AND, OR e NOT

AND exige que as **duas** condições sejam verdadeiras. **OR** basta uma. **NOT** inverte. Pergunta: quais romances foram publicados antes de 1900?

```sql
SELECT titulo, ano FROM livros WHERE ano < 1900 AND genero = 'Romance';
```

TELA DO LAB



1. As duas condições ligadas por **AND**.
2. Três livros satisfazem as duas ao mesmo tempo.

RESULTADO ESPERADO

titulo	ano
Dom Casmurro	1899
Memórias Póstumas de Brás Cubas	1881
O Primo Basílio	1878

Experimente também (faça junto e compare com o resultado esperado):

```sql
SELECT titulo, genero FROM livros WHERE genero = 'Infantil' OR ano < 1890;
```

RESULTADO ESPERADO

titulo	genero
Memórias Póstumas de Brás Cubas	Romance
Reinações de Narizinho	Infantil
O Primo Basílio	Romance
Caçadas de Pedrinho	Infantil

```sql
SELECT titulo, preco FROM livros WHERE preco BETWEEN 30 AND 40;
```

RESULTADO ESPERADO

titulo	preco
Dom Casmurro	39.90
A Hora da Estrela	34.50
Vidas Secas	36.00

```sql
SELECT titulo FROM livros WHERE autor_id IN (1, 3);
```

RESULTADO ESPERADO

titulo
Dom Casmurro
Memórias Póstumas de Brás Cubas
Capitães da Areia
Gabriela, Cravo e Canela

```sql
SELECT titulo FROM livros WHERE paginas IS NULL;
```

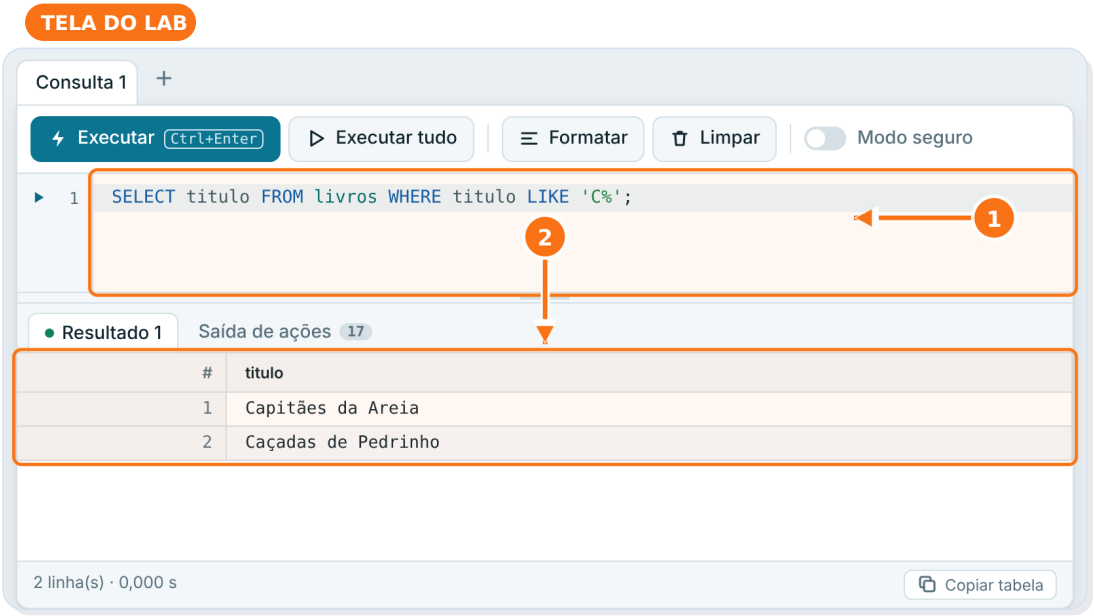
RESULTADO ESPERADO

titulo
Vidas Secas

15 Procurar por parte do texto: LIKE

O **LIKE** compara texto com um padrão. O símbolo **%** substitui “qualquer quantidade de letras”. `'C%'` = começa com C; `'%Pedrinho'` = termina com Pedrinho; `'%casa%'` = contém “casa” em qualquer posição.

```sql
SELECT titulo FROM livros WHERE titulo LIKE 'C%';
```



1. Padrão **'C%'**: títulos que começam com C.
2. Voltaram Capitães da Areia e Caçadas de Pedrinho.

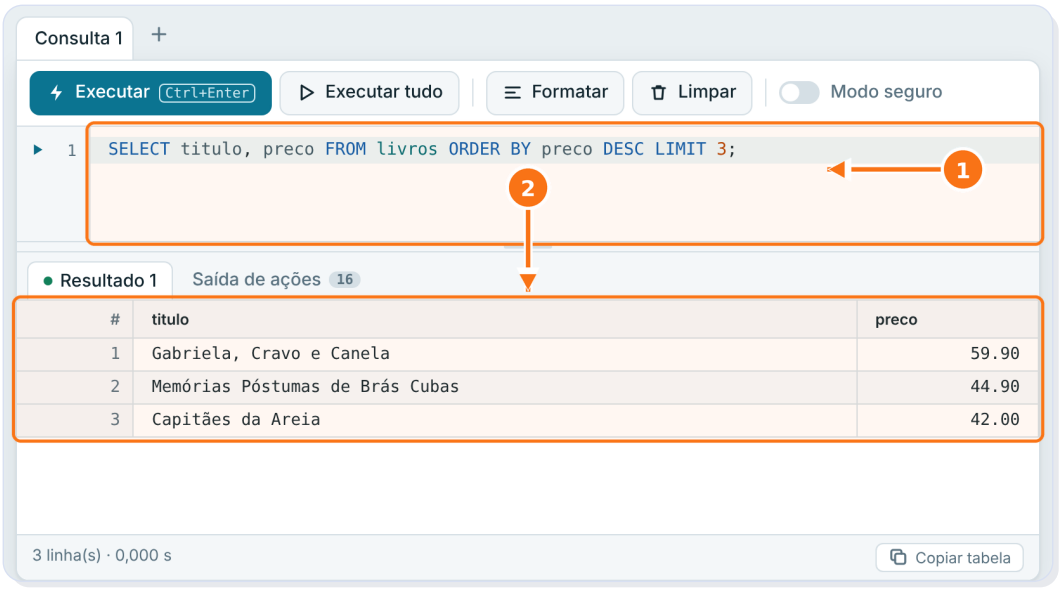
> **Dica.** O laboratório ignora diferença entre maiúsculas e minúsculas e acentos nas comparações de texto. Por isso 'capitaes' também encontraria “Capitães”.

16 Ordenar e limitar: ORDER BY e LIMIT

ORDER BY ordena o resultado (ASC = crescente, é o padrão; DESC = decrescente). **LIMIT** corta o resultado nas primeiras linhas. Juntos respondem: “quais são os 3 livros mais caros?”

```sql
SELECT titulo, preco FROM livros ORDER BY preco DESC LIMIT 3;
```

TELA DO LAB



1. **ORDER BY preco DESC**: do maior para o menor preço; **LIMIT 3**: só os três primeiros.
2. As três linhas mais caras.

RESULTADO ESPERADO

titulo	preco
Gabriela, Cravo e Canela	59.90
Memórias Póstumas de Brás Cubas	44.90
Capitães da Areia	42.00

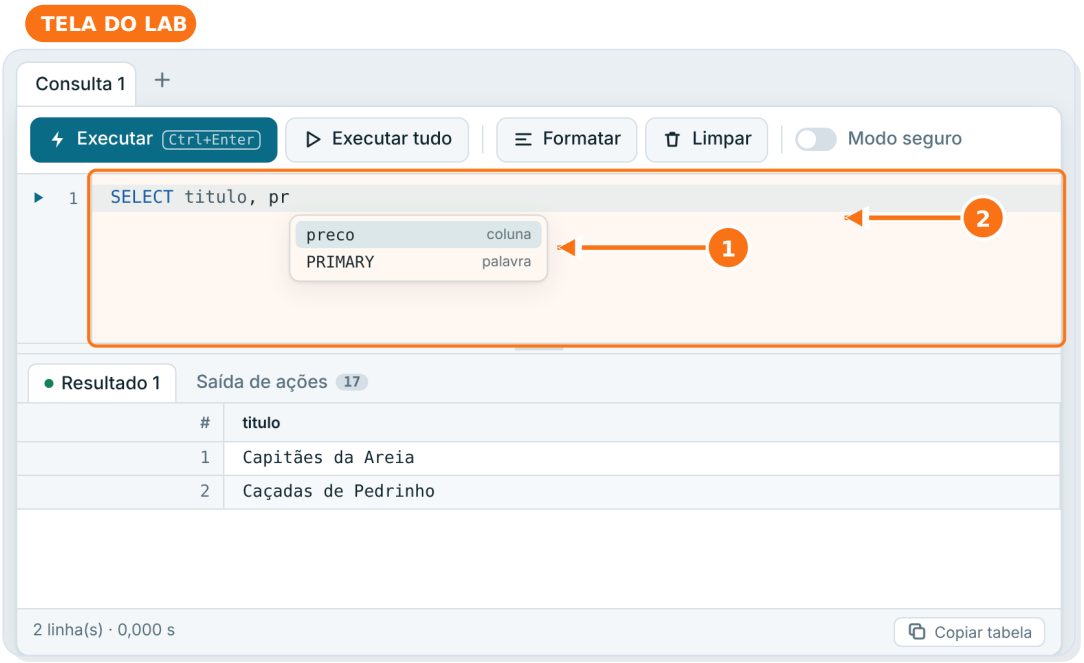
Outros recursos úteis

Recurso	Exemplo	O que faz
DISTINCT	`SELECT DISTINCT genero FROM livros;`	remove repetições: devolve Romance e Infantil, uma vez cada.
Apelido (AS)	`SELECT titulo AS livro, preco AS valor FROM livros;`	renomeia a coluna só na exibição.
Cálculo	`SELECT titulo, preco * 0.9 AS promo FROM livros;`	faz contas com as colunas (10% de desconto).
Funções de texto	`SELECT UPPER(titulo) FROM livros;`	coloca o título em maiúsculas.

17 Ajudas do editor: autocompletar e execução por linha

Enquanto você digita, o laboratório sugere palavras do SQL e nomes das suas tabelas e colunas. Experimente digitar `SELECT titulo, pr` e observe a lista:

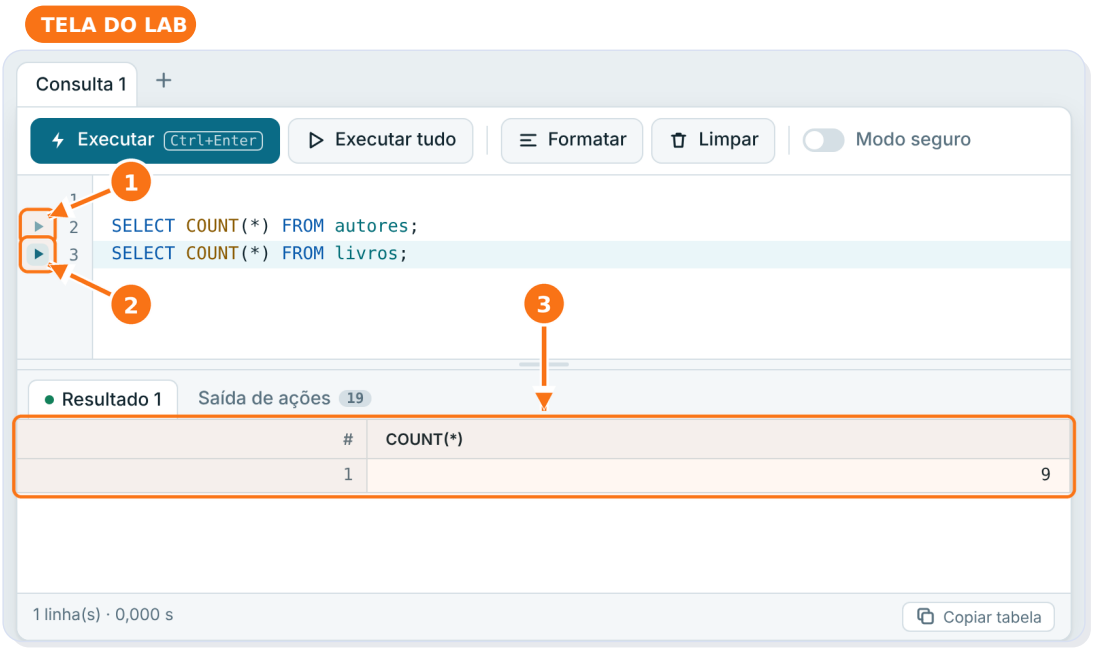
> **Faça junto.** Digite o texto acima **sem executar**. Use as setas ↑ ↓ para escolher e **Enter** ou **Tab** para aceitar a sugestão.



1. A **lista de sugestões** mostra **preco** (coluna da tabela livros).
2. O texto que você está digitando.

Executar um comando de cada vez

Quando há vários comandos no editor, o triângulo ▶ ao lado do número da linha executa só aquele comando:

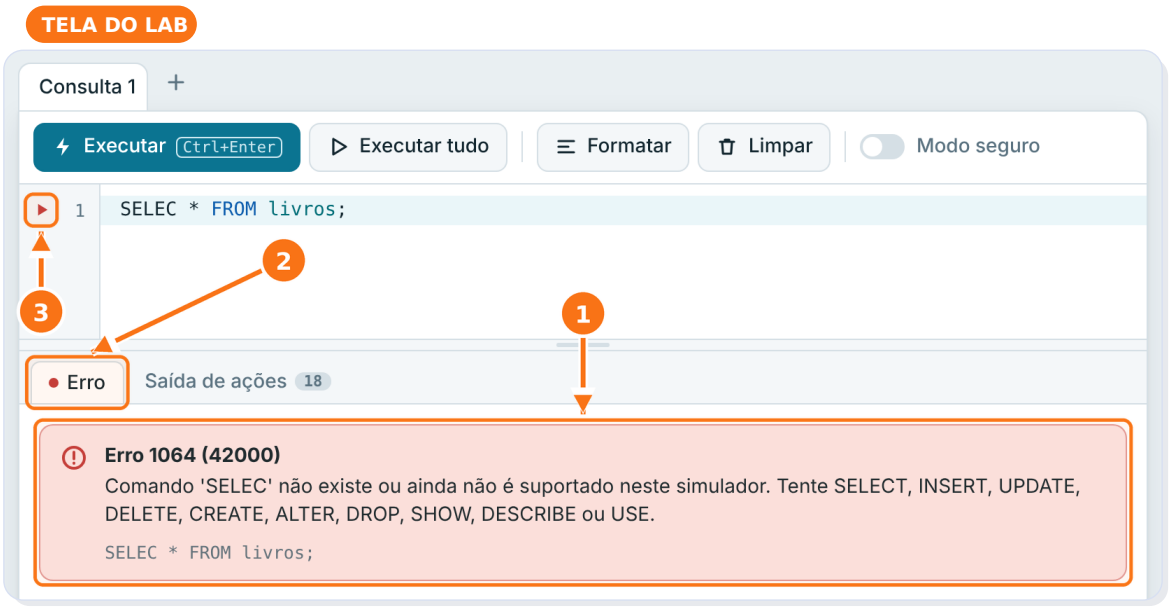


1. O ▶ da primeira linha executa só a contagem de autores.
2. O ▶ da segunda linha executa só a contagem de livros (foi o que clicamos).
3. O resultado: **9** livros. Experimente clicar nos dois ▶ e compare.

18 Quando dá erro

Errar é parte do processo. O laboratório explica o problema em português. Digite, de propósito, um comando com erro:

```sql
SELEC * FROM livros;
```



1. A **faixa vermelha** traz o número do erro (1064) e uma explicação: “SELEC não existe”, com sugestões.
2. A aba de resultado fica marcada como erro.
3. O ▶ da linha também indica o comando que falhou.

Erro	Quando acontece	O que fazer
1064	Erro de sintaxe: palavra escrita errada, vírgula ou parêntese faltando.	Releia o comando letra por letra.
1146	A tabela não existe (ou você esqueceu o USE).	Rode USE biblioteca; e SHOW TABLES;
1054	A coluna não existe.	Use DESCRIBE nome_da_tabela; para ver os nomes.
1046	Nenhum banco selecionado.	Rode USE biblioteca;

CAPÍTULO 7

Alterando e apagando dados

Corrigir um preço, remover um livro e entender as proteções do banco.

7

VOCÊ VAI APRENDER

- ✓ Corrigir linhas com UPDATE
- ✓ Apagar linhas com DELETE
- ✓ Por que o banco recusa apagar um autor que tem livros
- ✓ O Modo seguro e o ALTER TABLE

Cuidado. UPDATE e DELETE **sem WHERE** atuam em **todas as linhas da tabela**. Escreva sempre o WHERE antes de executar. Na vida real, um DELETE sem WHERE pode apagar o sistema inteiro de uma empresa.

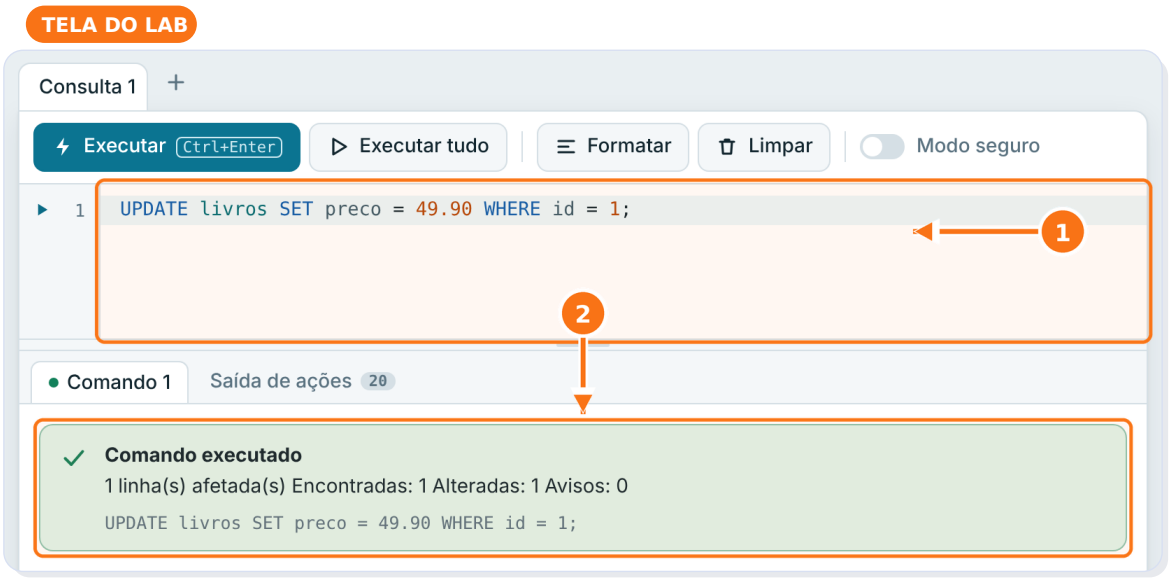
19 Corrigir um valor: UPDATE

O que faz: altera valores de linhas que já existem. **Para que serve:** corrigir erros e manter os dados atualizados. Vamos reajustar o preço de *Dom Casmurro*:

```sql
UPDATE livros SET preco = 49.90 WHERE id = 1;
```

CADA PEDAÇO DO COMANDO

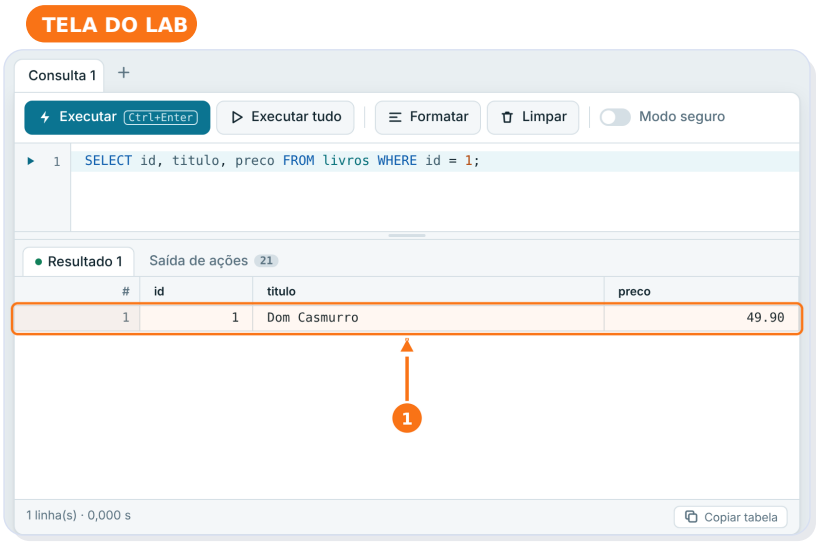
Pedaço	O que significa
`UPDATE livros`	“atualizar a tabela livros”.
`SET preco = 49.90`	“defina o preço como 49.90”. Para várias colunas: SET preco = 49.90, ano = 1900.
`WHERE id = 1`	**qual linha** mudar. Usar o id garante que só um livro será afetado.



1. O comando com **WHERE id = 1**.
2. **1 linha afetada**; “Encontradas: 1 · Alteradas: 1”.

Confira o resultado com um SELECT:

```sql
SELECT id, titulo, preco FROM livros WHERE id = 1;
```



1. O preço de Dom Casmurro agora é **49.90**.

20 Apagar linhas: DELETE

O que faz: remove linhas de uma tabela. **Para que serve:** eliminar registros que não devem mais existir. Antes, vamos ver a proteção da chave estrangeira. Tente apagar o autor 1 (Machado de Assis), que tem livros:

```sql
DELETE FROM autores WHERE id = 1;
```



1 **Erro 1451**: o banco recusou. Os livros 1 e 2 ainda apontam para esse autor; apagá-lo deixaria livros sem autor.

A saída correta é apagar primeiro os livros (as linhas “filhas”) e depois, se quiser, o autor. Vamos apagar apenas o livro 9, *Caçadas de Pedrinho*:

```sql
DELETE FROM livros WHERE id = 9;
```



1. O DELETE com WHERE id = 9.
2. **1 linha afetada**: o livro saiu da tabela.

> **Faça junto.** Faça os quatro comandos desta seção na ordem: UPDATE, SELECT de conferência, o DELETE do autor (para ver o erro) e o DELETE do livro. Os resultados dos próximos capítulos já consideram essas mudanças (8 livros, e Dom Casmurro a 49.90).

21 O Modo seguro

Para evitar acidentes, o MySQL Workbench traz o “safe updates”. O laboratório tem o mesmo recurso: o interruptor **Modo seguro**. Ligado, ele só deixa rodar UPDATE e DELETE que tenham WHERE em uma **coluna-chave** (como o id). Veja o que acontece ao tentar mudar todos os preços:

```sql
UPDATE livros SET preco = 10;
```



1. Ligue o interruptor **Modo seguro**.
2. **Erro 1175**: o comando foi barrado porque não tem WHERE com coluna-chave.

Importante. Com o Modo seguro desligado, esse mesmo comando **mudaria o preço de todos os livros** para 10. **Deixe o interruptor desligado após o teste**; os próximos exemplos não dependem dele.

Dica. Nos sistemas reais, antes de um UPDATE ou DELETE, rode um SELECT com o mesmo WHERE para ver exatamente quais linhas serão afetadas.

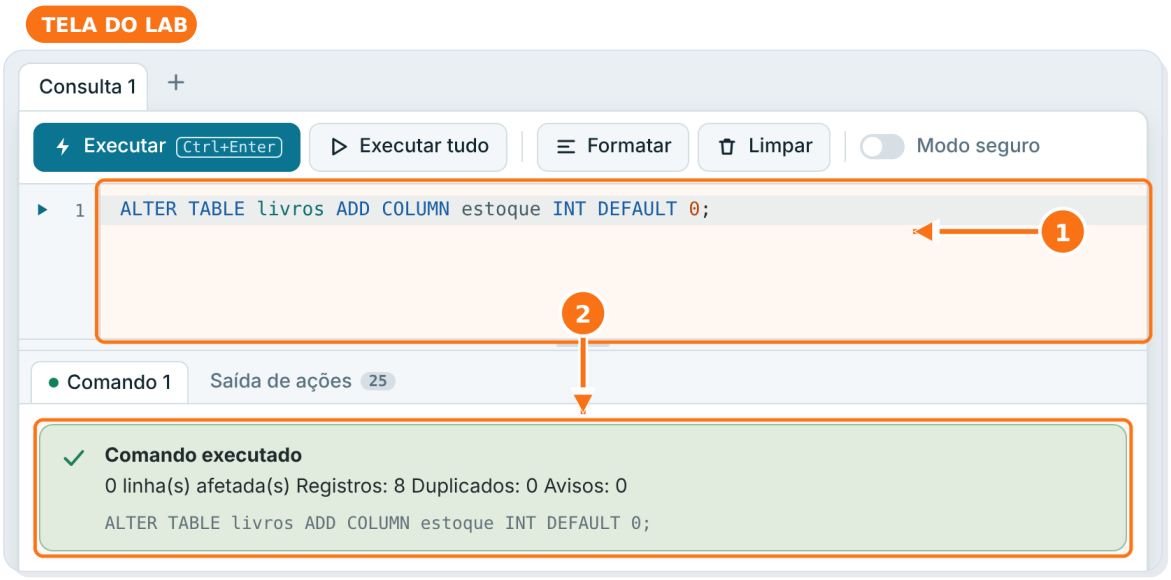
22 Alterar a estrutura: ALTER TABLE

O que faz: muda a estrutura de uma tabela que já existe. **Para que serve:** o projeto evolui e novas informações aparecem. Vamos adicionar o controle de estoque, sem perder os dados:

```sql
ALTER TABLE livros ADD COLUMN estoque INT DEFAULT 0;
```

CADA PEDAÇO DO COMANDO

Pedaço	O que significa
`ALTER TABLE livros`	“alterar a tabela livros”.
`ADD COLUMN estoque INT`	adiciona uma coluna nova chamada **estoque**, do tipo inteiro.
`DEFAULT 0`	valor inicial para as linhas já existentes: todas começam com estoque 0.



1. O comando ALTER TABLE.
2. Sucesso: “Registros: 8”, as 8 linhas existentes ganharam a nova coluna.

Outras formas do ALTER TABLE:

Comando	Efeito
`ALTER TABLE livros DROP COLUMN estoque;`	remove a coluna (e os dados dela).
`ALTER TABLE livros MODIFY titulo VARCHAR(120) NOT NULL;`	muda o tipo ou as regras de uma coluna.
`ALTER TABLE livros RENAME TO acervo;`	renomeia a tabela.

> **Pare e pense.** Qual é a diferença entre **UPDATE** e **ALTER TABLE**? O UPDATE muda *valores* (os dados); o ALTER muda a *forma* da tabela (a estrutura).

CAPÍTULO 8

Juntando tabelas com JOIN

A recompensa de separar os dados: perguntar sobre autores e livros ao mesmo tempo.

8

VOCÊ VAI APRENDER

- ✓ Entender por que o JOIN existe
- ✓ Escrever INNER JOIN com ON e apelidos de tabela
- ✓ Ler o resultado de uma junção
- ✓ Conhecer o LEFT JOIN

Lembra que a tabela **livros** guarda só o número do autor (autor_id)? Para mostrar o *nome* do autor ao lado do título, o banco precisa consultar as duas tabelas e "casar" cada livro com o seu autor. Isso é o **JOIN** (junção).

23 Mostrar título e autor

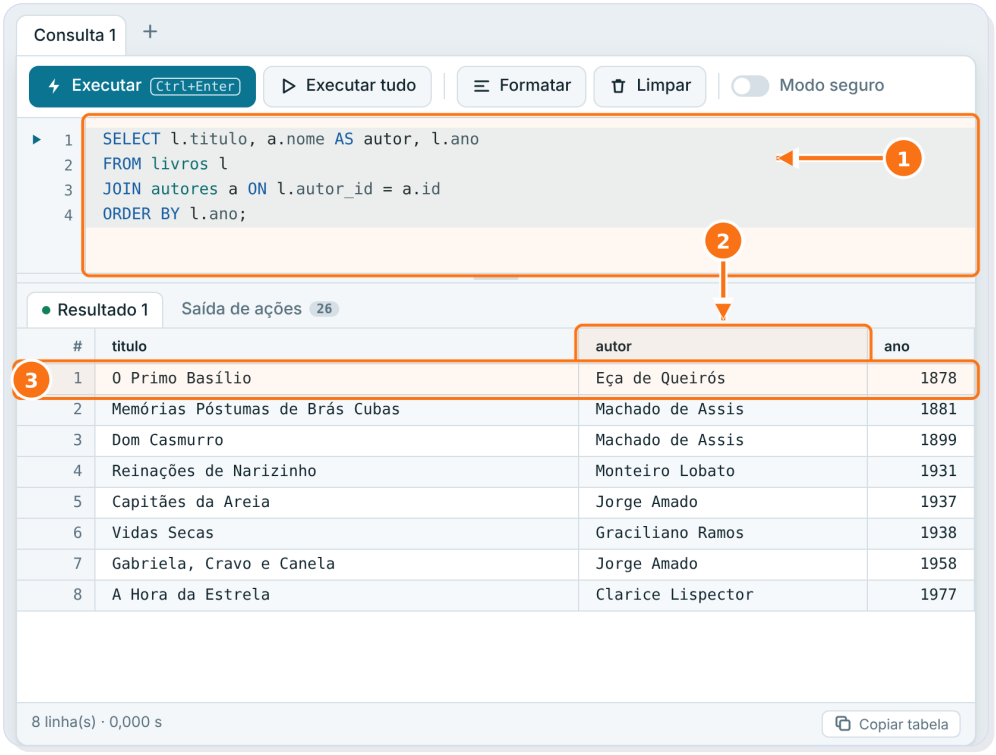
```sql
SELECT l.titulo, a.nome AS autor, l.ano
FROM livros l
JOIN autores a ON l.autor_id = a.id
ORDER BY l.ano;
```

CADA PEDAÇO DO COMANDO

Pedaço	O que significa
`FROM livros l`	tabela principal. **l** é um apelido: um nome curto para não repetir "livros".
`JOIN autores a`	junte com a tabela autores, apelidada de **a**.
`ON l.autor_id = a.id`	a **regra do casamento**: o autor_id do livro é igual ao id do autor.
`a.nome AS autor`	o prefixo **a.** diz de qual tabela é a coluna; AS dá um nome amigável.
`ORDER BY l.ano`	ordena os livros do mais antigo ao mais novo.

Faça junto. Digite a consulta completa (em várias linhas) e execute.

TELA DO LAB



1. A consulta usa as **duas tabelas** numa só.
2. Coluna **autor**: veio da tabela autores, pelo vínculo.
3. Cada linha junta um livro ao seu autor. Resultado: **8 linhas** (o livro 9 foi apagado).

RESULTADO ESPERADO

titulo	autor	ano
O Primo Basílio	Eça de Queirós	1878
Memórias Póstumas de Brás Cubas	Machado de Assis	1881
Dom Casmurro	Machado de Assis	1899
Reinações de Narizinho	Monteiro Lobato	1931
Capitães da Areia	Jorge Amado	1937
Vidas Secas	Graciliano Ramos	1938
Gabriela, Cravo e Canela	Jorge Amado	1958
A Hora da Estrela	Clarice Lispector	1977

Erro comum. **Erro 1052 (coluna ambígua)**: id existe nas duas tabelas. Quando duas tabelas têm colunas de mesmo nome, escreva sempre o prefixo: `l.id` ou `a.id`.

E os autores sem livros? (LEFT JOIN)

O JOIN comum (INNER JOIN) só mostra autores que têm livros. Para listar **todos** os autores, inclusive os que ainda não têm livro, use **LEFT JOIN**: ele mantém todas as linhas da tabela da esquerda e preenche com NULL quando não há par. Cadastre uma nova autora e teste:

```sql
INSERT INTO autores (nome, pais) VALUES ('Cecília Meireles', 'Brasil');
SELECT a.nome, l.titulo
FROM autores a
LEFT JOIN livros l ON l.autor_id = a.id
ORDER BY a.id;
```

RESULTADO ESPERADO

nome	titulo
Machado de Assis	Dom Casmurro
Machado de Assis	Memórias Póstumas de Brás Cubas
Clarice Lispector	A Hora da Estrela
Jorge Amado	Capitães da Areia
Jorge Amado	Gabriela, Cravo e Canela
Monteiro Lobato	Reinações de Narizinho
Graciliano Ramos	Vidas Secas
Eça de Queirós	O Primo Basílio
Cecília Meireles	NULL

Dica. Dica de leitura: JOIN = “só quem tem par dos dois lados”. LEFT JOIN = “todos da esquerda, com ou sem par”.

Pare e pense. Por que a Cecília Meireles aparece com NULL no título? Porque ela existe em **autores**, mas nenhum livro aponta para o id dela.

CAPÍTULO 9

Contando, somando e agrupando

Transforme linhas em respostas: quantos, qual a média, o maior, o total por grupo.

9

VOCÊ VAI APRENDER

- ✓ Funções COUNT, SUM, AVG, MIN e MAX
- ✓ Agrupar resultados com GROUP BY
- ✓ Filtrar grupos com HAVING
- ✓ Combinar JOIN e GROUP BY

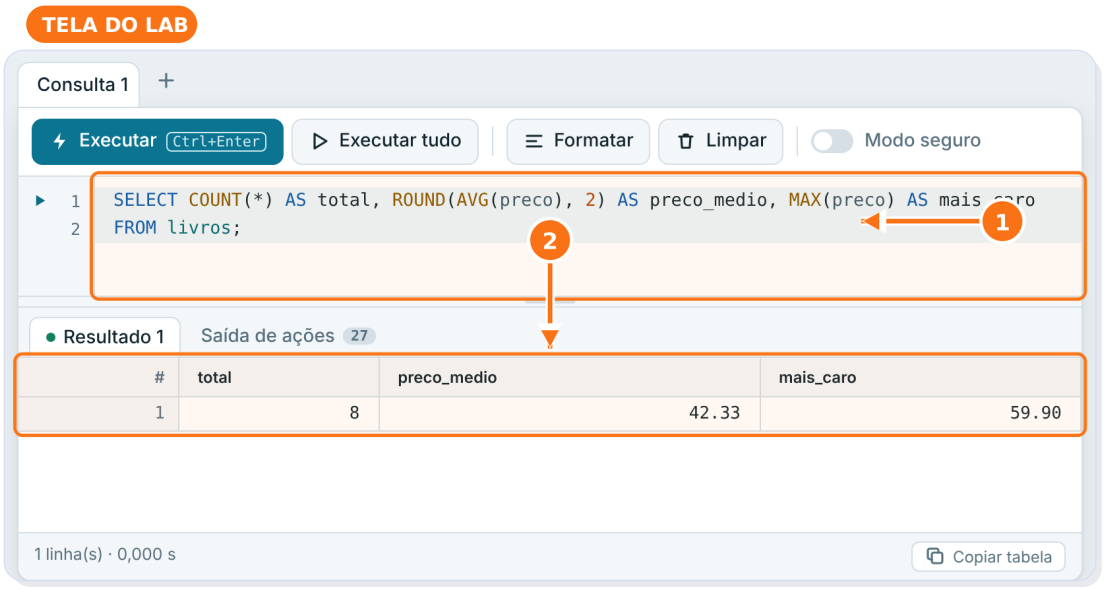
24 Funções de agregação

Uma **função de agregação** pega muitas linhas e devolve **um único valor**:

Função	Devolve
`COUNT(*)`	quantas linhas existem
`SUM(coluna)`	a soma dos valores (ignora NULL)
`AVG(coluna)`	a média dos valores
`MIN(coluna) / MAX(coluna)`	o menor / o maior valor
`ROUND(valor, 2)`	arredonda para 2 casas decimais

SQL

```sql
SELECT COUNT(*) AS total, ROUND(AVG(preco), 2) AS preco_medio, MAX(preco) AS mais_caro
FROM livros;
```



1. O comando calcula três valores de uma vez.
2. Uma única linha: **8** livros, preço médio **42.33**, o mais caro **59.90**.

Dica. Os valores de AVG, SUM e MAX consideram o estado atual da tabela. Como Dom Casmurro custa 49.90 agora e o livro 9 foi apagado, os números só batem com os da apostila se você seguiu o capítulo 7.

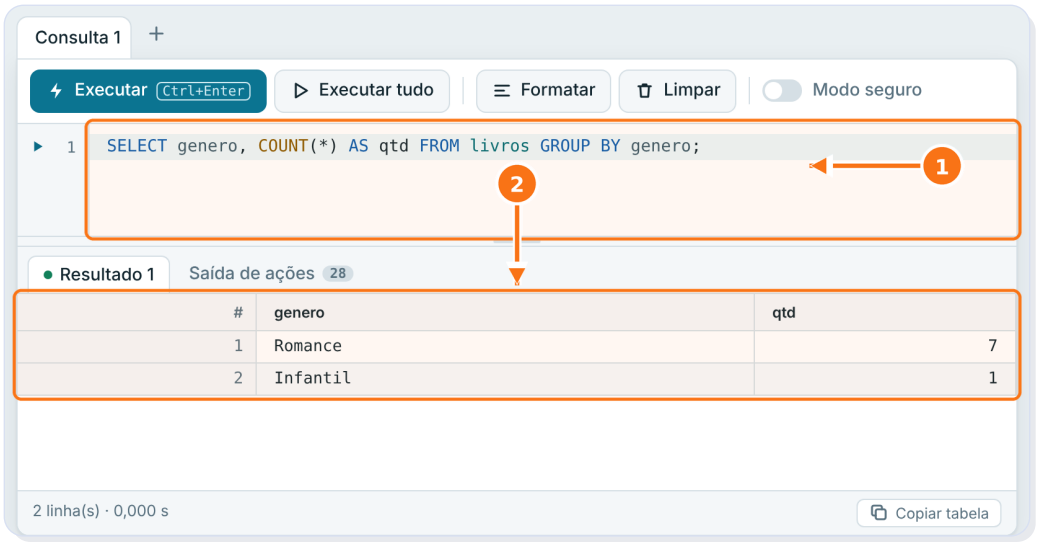
Pare e pense. O laboratório mostra **AVG(preco)** com mais casas decimais (por exemplo 42.325000). Por isso usamos ROUND para deixar o valor limpo.

25 Resumo por grupo: GROUP BY

O **GROUP BY** separa as linhas em grupos (um por valor da coluna) e aplica a função a cada grupo. Pergunta: quantos livros existem em cada gênero?

```sql
SELECT genero, COUNT(*) AS qtd FROM livros GROUP BY genero;
```

TELA DO LAB



1. **GROUP BY genero** cria um grupo por gênero.
2. Um resultado por grupo: 7 romances e 1 infantil.

> **Erro comum.** **Erro 1055**: toda coluna do SELECT que não está dentro de uma função de agregação precisa estar no GROUP BY. Se você pedir `SELECT genero, titulo ... GROUP BY genero`, o banco não sabe qual título mostrar.

Mais um exemplo, com soma e média por gênero:

```sql
SELECT genero, ROUND(AVG(preco), 2) AS media, SUM(paginas) AS total_pag
FROM livros
GROUP BY genero;
```

RESULTADO ESPERADO

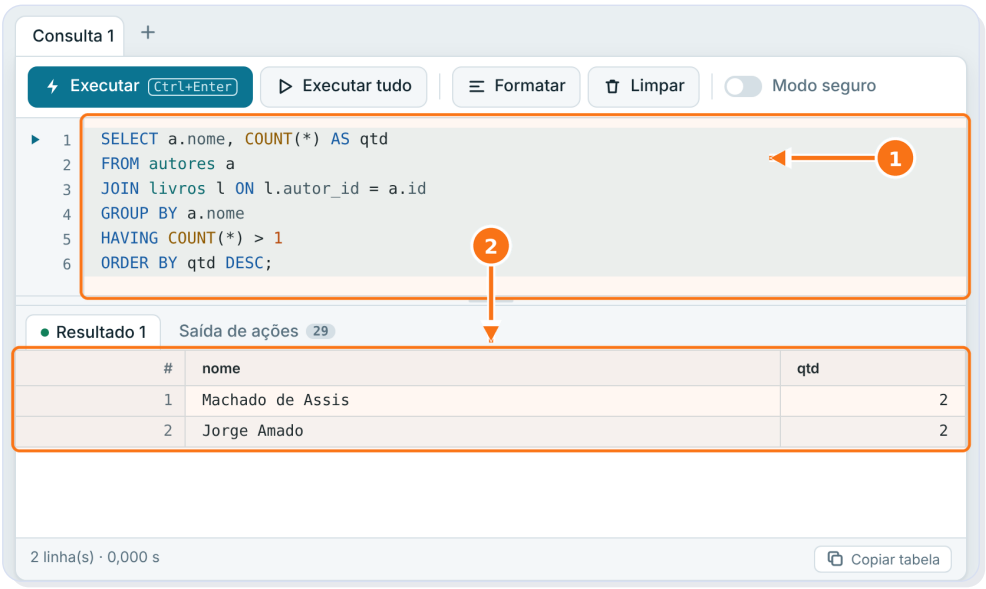
genero	media	total_pag
Romance	44.10	1816
Infantil	29.90	192

26 Filtrar grupos: HAVING, junto com JOIN

O **WHERE** filtra linhas *antes* do agrupamento; o **HAVING** filtra os grupos *depois*. Pergunta: quais autores têm mais de um livro cadastrado?

```sql
SELECT a.nome, COUNT(*) AS qtd
FROM autores a
JOIN livros l ON l.autor_id = a.id
GROUP BY a.nome
HAVING COUNT(*) > 1
ORDER BY qtd DESC;
```

TELA DO LAB



1. Junta autores e livros (JOIN) e agrupa por autor.
2. **HAVING COUNT(*) > 1** mantém só quem tem mais de um livro: Machado de Assis e Jorge Amado.

> **Importante.** Ordem das cláusulas num SELECT: `SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT`. Decore essa sequência: ela organiza qualquer consulta.

CAPÍTULO 10

Histórico e desafios

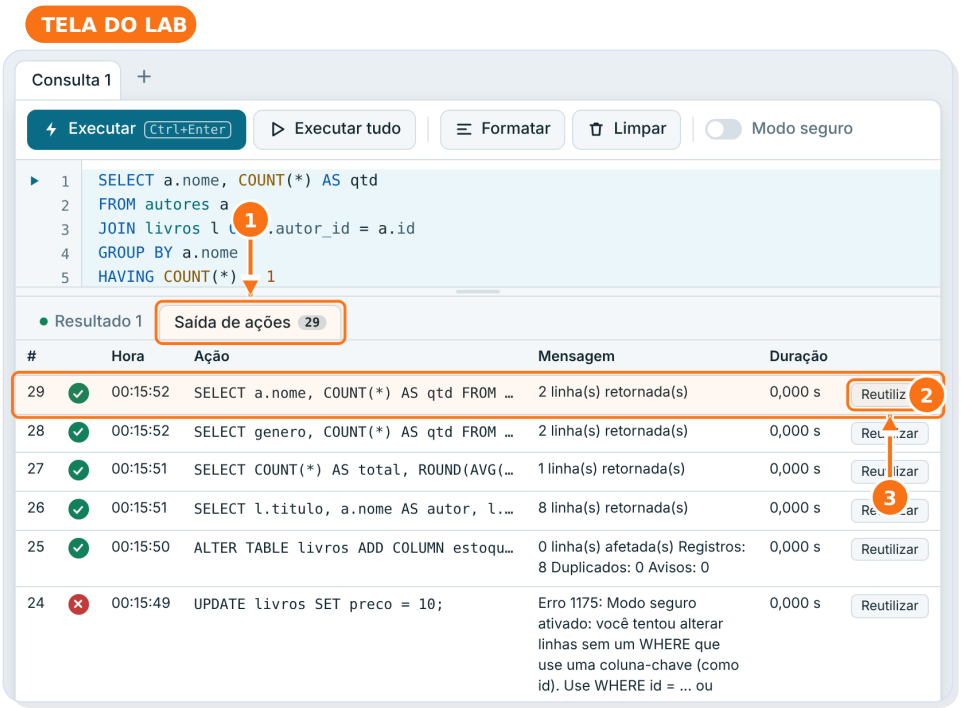
Revise o que você executou e teste o que aprendeu com perguntas prontas e correção automática.

VOCÊ VAI APRENDER

- ✓ Usar a Saída de ações para revisar e reutilizar comandos
- ✓ Resolver os desafios do laboratório

A saída de ações

Toda execução fica registrada na aba **Saída de ações**: hora, comando, mensagem e duração. É um histórico para você conferir o que deu certo e o que falhou.

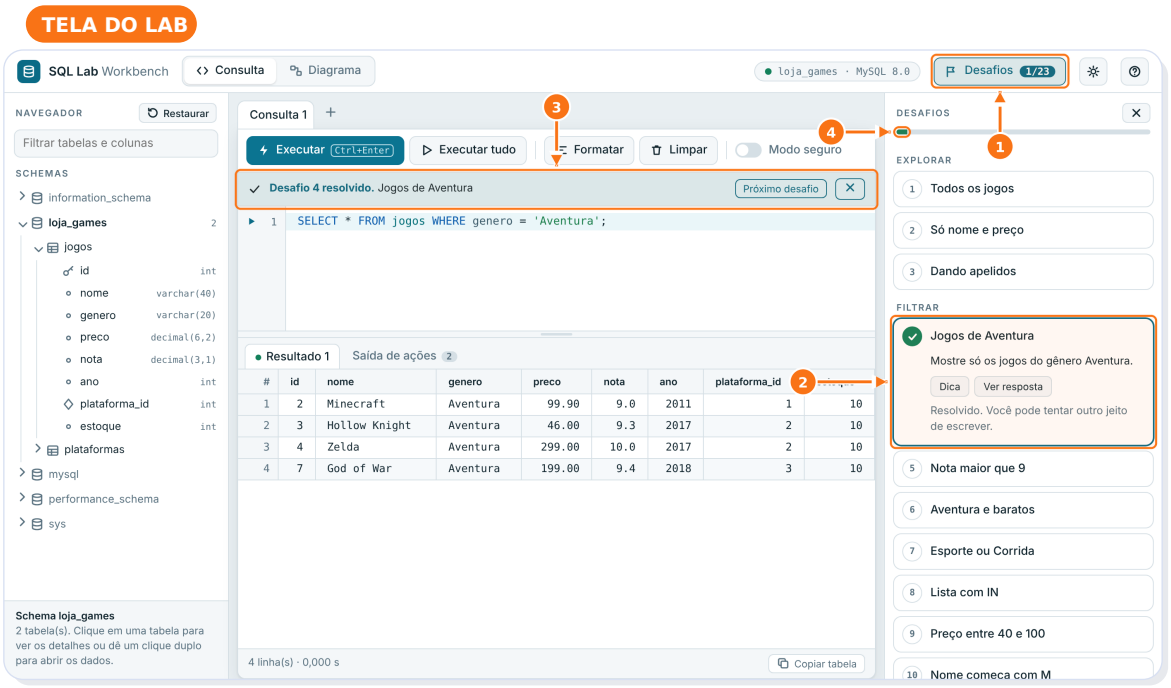


1. Clique na aba **Saída de ações**.
2. Cada linha é um comando executado (verde = sucesso, vermelho = erro).
3. **Reutilizar** devolve o comando ao editor para você alterar e rodar de novo.

Os desafios

O botão **Desafios** (no alto da tela) abre uma lista de 23 perguntas. Cada uma pede uma consulta sobre o banco de exemplo **loja_games**. O laboratório compara o seu resultado com o esperado e marca o desafio como resolvido.

> **Faça junto.** Clique em Desafios, escolha o primeiro item, leia o enunciado, digite a consulta e execute.



1. Abra o painel pelo botão **Desafios**.
2. Escolha uma pergunta da lista.
3. A **barra do objetivo** mostra o que foi pedido.
4. O **progresso** aumenta a cada desafio resolvido.

> **Dica.** Os desafios usam o banco **loja_games**. Se o seu banco ativo for outro, rode `USE loja_games;` antes. O seu banco biblioteca continua guardado.

CAPÍTULO 11

Projeto final: o banco da escola

Agora é com você: crie, do zero, um segundo banco, sem olhar os capítulos.

VOCÊ VAI APRENDER

- ✓ Aplicar sozinho tudo o que aprendeu
- ✓ Planejar tabelas e relacionamentos antes de escrever
- ✓ Conferir seu trabalho com as respostas do gabarito

Chegou a hora de caminhar com as próprias pernas. Você vai criar o banco de uma escola, com **cursos** e **alunos**. Primeiro planeje no papel:

Tabela	Colunas	Regras
`cursos`	id, nome, carga_horaria	id é chave primária com AUTO_INCREMENT; nome é obrigatório.
`alunos`	id, nome, email, nascimento, curso_id	id é chave primária; nome é obrigatório; curso_id é chave estrangeira para cursos(id); nascimento é do tipo DATE.

Missões

1. Crie o banco **escola** e entre nele.
2. Crie as tabelas **cursos** e **alunos**, nessa ordem. Confira com SHOW TABLES e DESCRIBE.
3. Cadastre 3 cursos e 6 alunos (deixe o e-mail de 2 alunos como NULL).
4. Liste todos os alunos.
5. Liste os alunos nascidos antes de junho de 2008, do mais velho ao mais novo.
6. Liste os alunos que não têm e-mail.
7. Mostre o nome de cada aluno com o nome do seu curso.
8. Mostre quantos alunos há em cada curso, do maior para o menor.
9. Corrija o e-mail de um aluno com UPDATE e confira com SELECT.
10. Abra o Diagrama e confira se a ligação entre as tabelas aparece.

Dica. Dica: use o apêndice A apenas para conferir, depois de tentar. Errar e corrigir ensina mais do que copiar a resposta.

Pare e pense. Se um dia a escola quiser guardar também as **notas** dos alunos, que tabela nova você criaria e como ligaria com a de alunos?

APÊNDICE

Gabarito do projeto final

Compare com a sua solução. Seu código pode ser diferente e ainda assim estar certo.

VOCÊ VAI APRENDER

- ✓ Conferir cada missão do projeto da escola

projeto_escola_1.sql

```sql
-- 1. banco
CREATE DATABASE escola;
USE escola;

-- 2. tabelas (cursos primeiro, porque alunos aponta para ela)
CREATE TABLE cursos (
  id INT NOT NULL AUTO_INCREMENT,
  nome VARCHAR(40) NOT NULL,
  carga_horaria INT,
  PRIMARY KEY (id)
);
CREATE TABLE alunos (
  id INT NOT NULL AUTO_INCREMENT,
  nome VARCHAR(60) NOT NULL,
  email VARCHAR(60),
  nascimento DATE,
  curso_id INT,
  PRIMARY KEY (id),
  FOREIGN KEY (curso_id) REFERENCES cursos(id)
);

-- 3. dados
INSERT INTO cursos (nome, carga_horaria) VALUES
  ('Informática', 1200), ('Administração', 900), ('Enfermagem', 1500);
INSERT INTO alunos (nome, email, nascimento, curso_id) VALUES
  ('Ana Lima', 'ana@email.com', '2008-03-15', 1),
  ('Bruno Souza', 'bruno@email.com', '2007-11-02', 1),
  ('Carla Dias', NULL, '2009-07-21', 2),
  ('Diego Alves', 'diego@email.com', '2008-01-30', 3),
  ('Elisa Rocha', 'elisa@email.com', '2007-05-09', 3),
  ('Fábio Lima', NULL, '2009-12-12', 1);
```

projeto_escola_2.sql

```sql
-- 4. todos os alunos
SELECT * FROM alunos;

-- 5. nascidos antes de junho de 2008, do mais velho ao mais novo
SELECT nome, nascimento FROM alunos
WHERE nascimento < '2008-06-01'
ORDER BY nascimento;

-- 6. sem e-mail
SELECT nome FROM alunos WHERE email IS NULL;

-- 7. aluno com o nome do curso
SELECT a.nome, c.nome AS curso
FROM alunos a
JOIN cursos c ON a.curso_id = c.id
ORDER BY a.nome;

-- 8. alunos por curso
SELECT c.nome, COUNT(*) AS alunos
FROM cursos c
JOIN alunos a ON a.curso_id = c.id
GROUP BY c.nome
ORDER BY alunos DESC;

-- 9. corrigir um e-mail
UPDATE alunos SET email = 'carla@email.com' WHERE id = 3;
SELECT id, nome, email FROM alunos WHERE id = 3;
```

RESULTADO ESPERADO

nome	alunos
Informática	3
Enfermagem	2
Administração	1

> **Dica.** A missão 8 deve mostrar Informática com 3 alunos, Enfermagem com 2 e Administração com 1.

APÊNDICE

Refazer o banco biblioteca

Perdeu o banco por ter recarregado a página? Use este roteiro para reconstruí-lo.

VOCÊ VAI APRENDER

- ✓ Reconstruir tudo com poucos comandos

Cole o texto abaixo no editor e clique em **Executar tudo** (Ctrl+Shift+Enter). Ele recria o banco, as tabelas e os dados até o capítulo 5.

biblioteca_completo.sql

```sql
CREATE DATABASE biblioteca;
USE biblioteca;

CREATE TABLE autores (
  id INT NOT NULL AUTO_INCREMENT,
  nome VARCHAR(50) NOT NULL,
  pais VARCHAR(30),
  PRIMARY KEY (id)
);

CREATE TABLE livros (
  id INT NOT NULL AUTO_INCREMENT,
  titulo VARCHAR(80) NOT NULL,
  genero VARCHAR(20),
  preco DECIMAL(6,2),
  ano INT,
  paginas INT,
  autor_id INT,
  PRIMARY KEY (id),
  FOREIGN KEY (autor_id) REFERENCES autores(id)
);

INSERT INTO autores (nome, pais) VALUES
  ('Machado de Assis', 'Brasil'),
  ('Clarice Lispector', 'Brasil'),
  ('Jorge Amado', 'Brasil'),
  ('Monteiro Lobato', 'Brasil'),
  ('Graciliano Ramos', 'Brasil'),
```

```
  ('Eça de Queirós', 'Portugal');

INSERT INTO livros (titulo, genero, preco, ano, paginas, autor_id) VALUES
  ('Dom Casmurro', 'Romance', 39.90, 1899, 256, 1),
  ('Memórias Póstumas de Brás Cubas', 'Romance', 44.90, 1881, 368, 1),
  ('A Hora da Estrela', 'Romance', 34.50, 1977, 88, 2),
  ('Capitães da Areia', 'Romance', 42.00, 1937, 280, 3),
  ('Gabriela, Cravo e Canela', 'Romance', 59.90, 1958, 424, 3),
  ('Reinações de Narizinho', 'Infantil', 29.90, 1931, 192, 4),
  ('Vidas Secas', 'Romance', 36.00, 1938, NULL, 5),
  ('O Primo Basílio', 'Romance', 41.50, 1878, 400, 6),
  ('Caçadas de Pedrinho', 'Infantil', 27.90, 1933, 120, 4);
```

APÊNDICE

Cola de comandos

Uma página para consultar sempre que tiver dúvida.

VOCÊ VAI APRENDER

✓ Lembrar rapidamente a sintaxe de cada comando

Para...	Use
Ver bancos	`SHOW DATABASES;`
Criar banco	`CREATE DATABASE nome;`
Entrar no banco	`USE nome;`
Ver tabelas	`SHOW TABLES;`
Ver estrutura	`DESCRIBE tabela;`
Criar tabela	`CREATE TABLE t (id INT NOT NULL AUTO_INCREMENT, ..., PRIMARY KEY (id));`
Inserir	`INSERT INTO t (col1, col2) VALUES (v1, v2);`
Consultar tudo	`SELECT * FROM t;`
Escolher colunas	`SELECT col1, col2 FROM t;`
Filtrar	`SELECT ... FROM t WHERE condição;`
Ordenar e limitar	`... ORDER BY col DESC LIMIT 3;`
Alterar dados	`UPDATE t SET col = v WHERE id = 1;`
Apagar linhas	`DELETE FROM t WHERE id = 1;`
Adicionar coluna	`ALTER TABLE t ADD COLUMN col INT;`
Juntar tabelas	`... FROM a JOIN b ON a.id = b.a_id`
Contar por grupo	`SELECT col, COUNT(*) FROM t GROUP BY col;`
Apagar tabela	`DROP TABLE t; (cuidado: apaga também os dados)`

Atalhos do laboratório

Atalho	Ação
Ctrl + Enter	executa o comando onde o cursor está
Ctrl + Shift + Enter	executa todos os comandos
Enter ou Tab (na lista)	aceita a sugestão do autocompletar
Esc	fecha a lista de sugestões
▶ ao lado da linha	executa aquele comando

APÊNDICE

Glossário

As palavras que você aprendeu, em ordem alfabética.

VOCÊ VAI APRENDER

- ✓ Rever o significado de cada termo

Termo	Significado
AUTO_INCREMENT	numeração automática de uma coluna.
Banco de dados	conjunto organizado de tabelas de um sistema.
Chave estrangeira (FK)	coluna que aponta para a chave primária de outra tabela.
Chave primária (PK)	coluna que identifica cada linha de modo único.
Coluna	uma característica das linhas da tabela.
Consulta (query)	pergunta feita ao banco, em geral com SELECT.
DDL	comandos que definem a estrutura (CREATE, ALTER, DROP).
DML	comandos que mexem nos dados (INSERT, UPDATE, DELETE).
JOIN	junção de tabelas por colunas relacionadas.
Linha (registro)	uma ocorrência completa guardada na tabela.
NOT NULL	regra: a coluna não aceita valor vazio.
NULL	ausência de valor.
SGBD	programa que gerencia bancos de dados (MySQL, por exemplo).
SQL	linguagem usada para conversar com o banco.
Tabela	estrutura de linhas e colunas sobre um assunto.
Tipo de dado	define o que a coluna pode guardar (INT, VARCHAR, DECIMAL, DATE).